Name:Nivass C R

Roll no:241801193

Ex1: Data Preprocessing and Cleaning

Problem Statement:

A telecommunications company is experiencing customer churn (customers leaving the service). Before building a machine learning model to predict churn, the data science team must clean and preprocess the customer data to ensure high-quality input for analysis. You are a Data Scientist working for the company. Your task is to preprocess the customer dataset by handling missing values, removing inconsistencies, transforming categorical features, scaling numerical attributes, and visualizing important relationships.

Learning Objective:

The objective of this experiment is to enable students to understand and perform the essential steps involved in data preprocessing. This experiment will provide a strong foundation for preparing high-quality data for subsequent machine learning and predictive analytics tasks.

Tasks to Perform:

1. Import the required Python libraries.

2. Load the Telco Customer Churn Dataset into a Pandas DataFrame. And display First 10 records, Last 10 records, Shape of the dataset, Column names.

3. Explore the Dataset and display Dataset information, Summary statistics, Number of missing values, Number of duplicate records

4. Clean the data a. Remove duplicate customer records.

b. Replace missing values

i. Numerical columns → Median

ii. Categorical columns → Mode

c. Verify that no missing values remain.

5. Encode categorical columns: Gender, Partner, Dependents, Contract, PaymentMethod, InternetService, Churn. Use appropriate encoding techniques.

6. Standardise the numerical features: Tenure, MonthlyCharges, TotalCharges, using StandardScaler.

7. Plot Distribution of MonthlyCharges, Distribution of Tenure.

8. Create a Count Plot showing Contract vs Churn.

9. Draw a Box Plot of MonthlyCharges grouped by Churn.

10. Display the Correlation Heatmap of all numerical features.

```python
# Experiment - 1
# Nivass C R - 241801193
# Step 1
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler
```

```python
# Experiment - 1
# Nivass C R - 241801193
# Step 2
from google.colab import files
uploaded = files.upload()
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
print("\nDataset Imported Successfully")
print("First 10 Records",df.head(10))
print("Last 10 Records",df.tail(10))
print("Shape of Dataset",df.shape)
print("Column Names",df.columns)
```

```
7036  7750-EYXWZ  Female              0      No         No      12
7037  2569-WGERO  Female              0      No         No      72
7038  6840-RESVB    Male              0     Yes        Yes      24
7039  2234-XADUH  Female              0     Yes        Yes      72
7040  4801-JZAZL  Female              0     Yes        Yes      11
7041  8361-LTMKD    Male              1     Yes         No       4
7042  3186-AJIEK    Male              0      No         No      66

     PhoneService     MultipleLines InternetService       OnlineSecurity  ...  \
7033          Yes               No     Fiber optic                   No  ...
7034          Yes              Yes     Fiber optic                  Yes  ...
7035          Yes               No     Fiber optic                   No  ...
7036           No  No phone service             DSL                   No  ...
7037          Yes               No              No  No internet service  ...
7038          Yes              Yes             DSL                  Yes  ...
7039          Yes              Yes     Fiber optic                   No  ...
7040           No  No phone service             DSL                  Yes  ...
7041          Yes              Yes     Fiber optic                   No  ...
7042          Yes               No     Fiber optic                  Yes  ...

         DeviceProtection          TechSupport          StreamingTV  \
7033                   No                   No                   No
7034                  Yes                   No                  Yes
7035                   No                   No                  Yes
7036                  Yes                  Yes                  Yes
7037  No internet service  No internet service  No internet service
7038                  Yes                  Yes                  Yes
7039                  Yes                   No                  Yes
7040                   No                   No                   No
```

```python
# Nivass C R - 241801193
# Step 3

print("\nDataset Information",df.info())

print("\nSummary Statistics",df.describe())

print("\nMissing Values",df.isnull().sum())

print("\nDuplicate Records",df.duplicated().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB

Dataset Information None

Summary Statistics       SeniorCitizen       tenure  MonthlyCharges
count     7043.000000  7043.000000      7043.000000
mean         0.162147    32.371149        64.761692
std          0.368612    24.559481        30.090047
min          0.000000     0.000000        18.250000
25%          0.000000     9.000000        35.500000
50%          0.000000    29.000000        70.350000
75%          0.000000    55.000000        89.850000
max          1.000000    72.000000       118.750000

Missing Values customerID          0
gender             0
SeniorCitizen      0
Partner            0
Dependents         0
tenure             0
PhoneService       0
```

```
# Nivass C R - 241801193
# Step 4a
print("Duplicate Records Before:", df.duplicated().sum())
df.drop_duplicates(inplace=True)

print("Duplicate Records After:", df.duplicated().sum())
```

```
Duplicate Records Before: 0
Duplicate Records After: 0
```

```
# Nivass C R - 241801193
# Step 4b

numerical_columns = df.select_dtypes(include=["int64", "float64"]).columns

for col in numerical_columns:
    df[col].fillna(df[col].median(), inplace=True)

categorical_columns = df.select_dtypes(include=["object"]).columns

for col in categorical_columns:
    df[col].fillna(df[col].mode()[0], inplace=True)
```

```
/tmp/ipykernel_3442/1638646499.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df[col].fillna(df[col].median(), inplace=True)
/tmp/ipykernel_3442/1638646499.py:13: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df[col].fillna(df[col].mode()[0], inplace=True)
```

```
# Nivass C R - 241801193
# Step 4c

print("Missing Values After Cleaning")
print(df.isnull().sum())
```

```
Missing Values After Cleaning
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

```
# Nivass C R - 241801193
# Step 5

le = LabelEncoder()

columns = ["gender", "Partner", "Dependents", "Contract",
           "PaymentMethod", "InternetService", "Churn"]

for col in columns:
    df[col] = le.fit_transform(df[col])

print("Encoding Completed")
print(df[columns].head())
```

```
Encoding Completed
   gender  Partner  Dependents  Contract  PaymentMethod  InternetService  \
0       0        1           0         0              2                0
1       1        0           0         1              3                0
2       1        0           0         0              3                0
3       1        0           0         1              0                0
4       0        0           0         0              2                1

   Churn
0      0
1      0
2      1
3      0
4      1
```

```python
# Nivass C R - 241801193
# Step 5

le = LabelEncoder()

columns = ["gender", "Partner", "Dependents", "Contract",
           "PaymentMethod", "InternetService", "Churn"]

for col in columns:
    df[col] = le.fit_transform(df[col])

print("Encoding Completed")
print(df[columns].head())
```

```
Encoding Completed
   gender  Partner  Dependents  Contract  PaymentMethod  InternetService  \
0       0        1           0         0              2                0
1       1        0           0         1              3                0
2       1        0           0         0              3                0
3       1        0           0         1              0                0
4       0        0           0         0              2                1

   Churn
0      0
1      0
2      1
3      0
4      1
```

```python
# Nivass C R - 241801193
# Step 6
scaler = StandardScaler()

cols = ["tenure", "MonthlyCharges", "TotalCharges"]

# Convert 'TotalCharges' to numeric, coercing errors to NaN
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

# Fill any NaN values that resulted from coercion with the median
df["TotalCharges"].fillna(df["TotalCharges"].median(), inplace=True)

df[cols] = scaler.fit_transform(df[cols])

print(df[cols].head())
```

```
     tenure  MonthlyCharges  TotalCharges
0 -1.277445       -1.160323     -0.994242
1  0.066327       -0.259629     -0.173244
2 -1.236724       -0.362660     -0.959674
3  0.514251       -0.746535     -0.194766
4 -1.236724        0.197365     -0.940470
/tmp/ipykernel_3442/2027301524.py:12: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

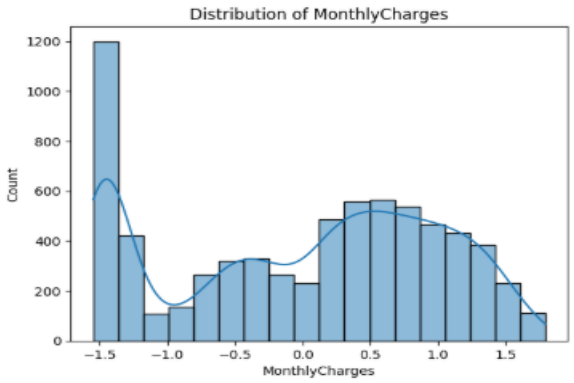

  df["TotalCharges"].fillna(df["TotalCharges"].median(), inplace=True)
```

```python
# Nivass C R - 241801193
# Step 7a

sns.histplot(df["MonthlyCharges"], kde=True)

plt.title("Distribution of MonthlyCharges")

plt.show()
```

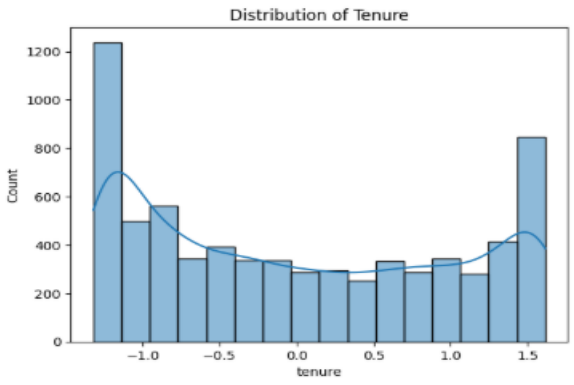



```python
# Nivass C R - 241801193
# Step 7b

sns.histplot(df["tenure"], kde=True)

plt.title("Distribution of Tenure")

plt.show()
```

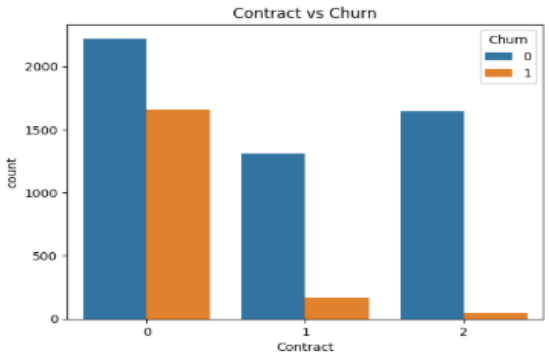


```
# Nivass C R - 241801193
# Step 8

sns.countplot(x="Contract", hue="Churn", data=df)

plt.title("Contract vs Churn")

plt.show()
```

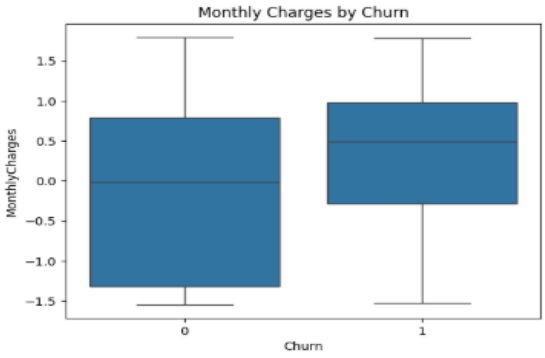



```
# Nivass C R - 241801193
# Step 9

sns.boxplot(x="Churn", y="MonthlyCharges", data=df)

plt.title("Monthly Charges by Churn")

plt.show()
```



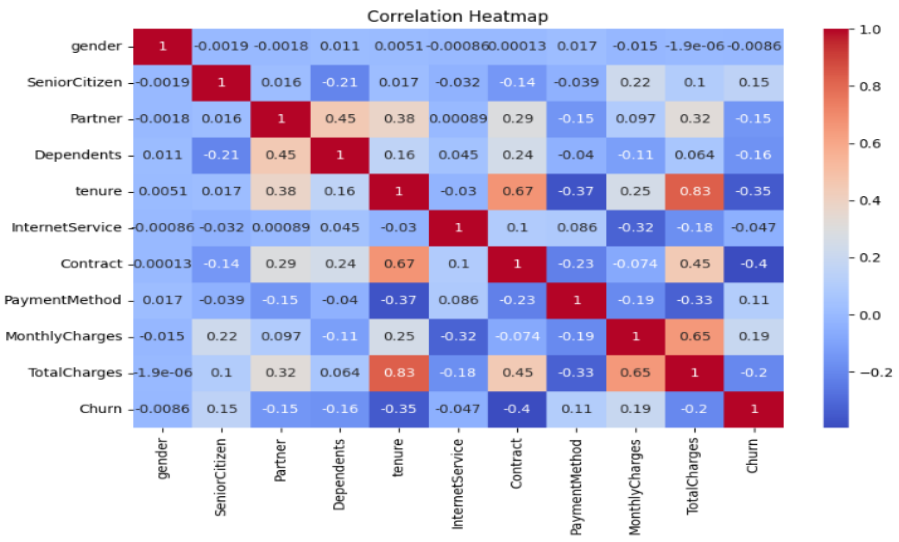

```
# Nivass C R - 241801193
# Step 10

plt.figure(figsize=(10,6))

sns.heatmap(df.corr(numeric_only=True),
            annot=True,
            cmap="coolwarm")

plt.title("Correlation Heatmap")

plt.show()
```



Ex2: Data Discovery and Preparation

Learning Objective: The objective of this experiment is to perform data discovery and exploratory data analysis on a real-world dataset.

Question: A telecommunications company has observed an increase in customer churn, leading to revenue loss. The management wants to understand customer behavior and identify patterns before developing a machine learning model to predict churn.

As a Data Analyst, your responsibility is to discover meaningful insights from the dataset, identify data quality issues, perform exploratory data analysis, detect outliers, and prepare the dataset for predictive analytics.

Tasks to Perform:

1. Reuse Ex:1 Steps 1- 6, 10.

2. Analyze the distribution of: Customer Churn, Gender, Contract Type, Payment Method.

3. Visualize the distribution of: MonthlyCharges, TotalCharges Tenure using Histograms and Boxplots

. 4. Detect outliers in numerical features using the Interquartile Range (IQR) method.

5. Create suitable visualizations (Count Plot, Box Plot, Histogram, or Scatter Plot) to study the relationship between: Contract Type and Churn, Internet Service and Churn, Monthly Charges and Churn, Tenure and Churn.

6. Do the below analytical questions: • Which contract type has the highest customer churn?

• Which payment method is most frequently used by customers who churn?

• Do customers with shorter tenure tend to churn more often?

• Which customer group has the highest monthly charges? • Which Internet Service category experiences the highest churn?

• Is there any relationship between monthly charges and churn?

• Which features appear to be the strongest indicators of customer churn?

7. Split the cleaned dataset into Training (80%) and Testing (20%) datasets. 8. Save the final processed dataset as a new CSV file named CustomerChurn_Preprocessed.csv.

```
#Nivass C R - 241801193
#EX-2
#step 1

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
```

```
#Nivass C R - 241801193
#step 2

from google.colab import files
import pandas as pd

uploaded = files.upload()

filename = next(iter(uploaded))
```

```
Choose files No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Titanic-Dataset.csv to Titanic-Dataset (1).csv
```

```
#Nivass C R - 241801193
#EX-2
#step 2
df = pd.read_csv(filename)

df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

```
#Nivass C R - 241801193
#EX-2
#step 3

# (a) Shape
df.shape

# (b) Information
df.info()

# (c) Describe
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
#Nivass C R - 241801193
#EX-2
#step 4

imputer = SimpleImputer(strategy='mean')

df['Age'] = imputer.fit_transform(df[['Age']])

print(df['Age'].head())
```

```
0    22.0
1    38.0
2    26.0
3    35.0
4    35.0
Name: Age, dtype: float64
```

```python
#Nivass C R - 241801193
#EX-2
#step 5

df['Cabin'] = df['Cabin'].fillna("Unknown")
print(df['Cabin'].head(10))
```

```
0    Unknown
1        C85
2    Unknown
3       C123
4    Unknown
5    Unknown
6        E46
7    Unknown
8    Unknown
9    Unknown
Name: Cabin, dtype: object
```

```python
#Nivass C R - 241801193
#EX-2
#step 6

mode_value = df['Embarked'].mode()[0]

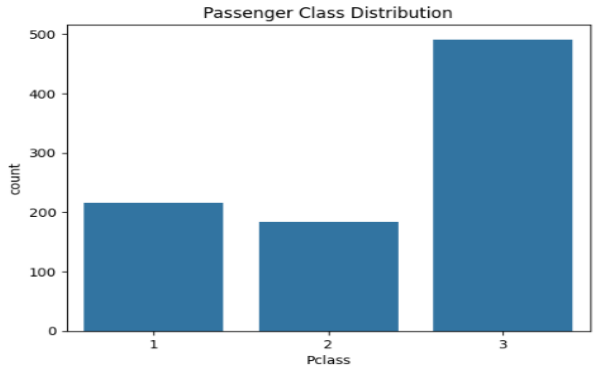
df['Embarked'] = df['Embarked'].fillna(mode_value)

print("Mode:", mode_value)
print(df['Embarked'].head(10))
```

```
Mode: S
0    S
1    C
2    S
3    S
4    S
5    Q
6    S
7    S
8    S
9    C
Name: Embarked, dtype: object
```

```python
#Nivass C R - 241801193
#EX-2
#step 7

sns.countplot(x='Pclass', data=df)
plt.title("Passenger Class Distribution")
plt.show()
```




```python
#Nivass C R - 241801193
#EX-2
#step 8

female_survived = df[(df['Sex']=="female") & (df['Survived']==1)]

print(female_survived['Name'])
```

```
1      Cumings, Mrs. John Bradley (Florence Briggs Th...
2                                  Heikkinen, Miss. Laina
3           Futrelle, Mrs. Jacques Heath (Lily May Peel)
8      Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)
9                    Nasser, Mrs. Nicholas (Adele Achem)
                             ...
874                Abelson, Mrs. Samuel (Hannah Wizosky)
875                     Najib, Miss. Adele Kiamie "Jane"
879         Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)
880          Shelley, Mrs. William (Imanita Parrish Hall)
887                          Graham, Miss. Margaret Edith
Name: Name, Length: 233, dtype: object
```

```python
#Nivass C R - 241801193
#EX-2
#step 9
result = df[(df['Pclass']==3) & (df['Age']<18)]
print(result)
```

```
     PassengerId  Survived  Pclass                                      Name  \
7              8         0       3                Palsson, Master. Gosta Leonard
10            11         1       3               Sandstrom, Miss. Marguerite Rut
14            15         0       3       Vestrom, Miss. Hulda Amanda Adolfina
16            17         0       3                       Rice, Master. Eugene
22            23         1       3               McGowan, Miss. Anna "Annie"
..           ...       ...     ...                                       ...
844          845         0       3                       Culumovic, Mr. Jeso
850          851         0       3  Andersson, Master. Sigvard Harald Elias
852          853         0       3                    Boulos, Miss. Nourelain
869          870         1       3           Johnson, Master. Harold Theodor
875          876         1       3          Najib, Miss. Adele Kiamie "Jane"

        Sex   Age  SibSp  Parch    Ticket     Fare    Cabin Embarked
7      male   2.0      3      1    349909  21.0750  Unknown        S
10   female   4.0      1      1   PP 9549  16.7000       G6        S
14   female  14.0      0      0    350406   7.8542  Unknown        S
16     male   2.0      4      1    382652  29.1250  Unknown        Q
22   female  15.0      0      0    330923   8.0292  Unknown        Q
..      ...   ...    ...    ...       ...      ...      ...      ...
844    male  17.0      0      0    315090   8.6625  Unknown        S
850    male   4.0      4      2    347082  31.2750  Unknown        S
852  female   9.0      1      1      2678  15.2458  Unknown        C
869    male   4.0      1      1    347742  11.1333  Unknown        S
875  female  15.0      0      0      2667   7.2250  Unknown        C

[78 rows x 12 columns]
```

```python
#Nivass C R - 241801193
#EX-2
#step 10
result = df[(df['Pclass']==1) & (df['Age']>40)]
print(result)
```

```
     PassengerId  Survived  Pclass  \
6              7         0       1
11            12         1       1
35            36         0       1
52            53         1       1
54            55         0       1
..           ...       ...     ...
856          857         1       1
857          858         1       1
862          863         1       1
871          872         1       1
879          880         1       1

                                                  Name     Sex   Age  SibSp  \
6                                  McCarthy, Mr. Timothy J    male  54.0      0
11                               Bonnell, Miss. Elizabeth  female  58.0      0
35                            Holverson, Mr. Alexander Oskar    male  42.0      1
52                 Harper, Mrs. Henry Sleeper (Myna Haxtun)  female  49.0      1
54                           Ostby, Mr. Engelhart Cornelius    male  65.0      0
..                                                      ...     ...   ...    ...
856            Wick, Mrs. George Dennick (Mary Hitchcock)  female  45.0      1
```

```python
#Nivass C R - 241801193
#EX-2
#step 11

result = df[(df['Pclass']==1) &
            (df['Age']>40) &
            (df['Survived']==1)]

print(result)
```

```
     621                          Kimball, Mr. Edwin Nelson Jr    male  42.0      1
     630                 Barkworth, Mr. Algernon Henry Wilson    male  80.0      0
     645                              Harper, Mr. Henry Sleeper    male  48.0      1
     647                   Simonius-Blumer, Col. Oberst Alfons    male  56.0      0
     660                        Frauenthal, Dr. Henry William    male  50.0      2
     707                     Calderhead, Mr. Edward Pennington    male  42.0      0
     712                               Taylor, Mr. Elmer Zebley    male  48.0      1
     765                   Hogeboom, Mrs. John C (Anna Andrews)  female  51.0      1
     779  Robert, Mrs. Edward Scott (Elisabeth Walton Mc...  female  43.0      0
     796                          Leader, Dr. Alice (Farnham)  female  49.0      0
     820  Hays, Mrs. Charles Melville (Clara Jennings Gr...  female  52.0      1
     829            Stone, Mrs. George Nelson (Martha Evelyn)  female  62.0      0
     856           Wick, Mrs. George Dennick (Mary Hitchcock)  female  45.0      1
     857                               Daly, Mr. Peter Denis    male  51.0      0
     862  Swift, Mrs. Frederick Joel (Margaret Welles Ba...  female  48.0      0
     871   Beckwith, Mrs. Richard Leonard (Sallie Monypeny)  female  47.0      1
     879       Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)  female  56.0      0

          Parch    Ticket      Fare    Cabin Embarked
     11       0    113783   26.5500     C103        S
     52       0  PC 17572   76.7292      D33        C
     187      0    111428   26.5500  Unknown        S
     194      0  PC 17610   27.7208       B4        C
     195      0  PC 17569  146.5208      B80        C
     268      1  PC 17582  153.4625     C125        S
     275      0     13502   77.9583       D7        S
     299      1  PC 17558  247.5208  B58 B60        C
     337      0     16966  134.5000      E40        C
     366      0    110813   75.2500      D37        C
     380      0  PC 17757  227.5250  Unknown        C
     449      0    113786   30.5000     C104        S
     453      0     17453   89.1042      C92        C
     460      0     19952   26.5500      E12        S
     496      0     36947   78.2667      D20        C
     513      0  PC 17603   59.4000  Unknown        C
     523      1    111361   57.9792      B18        C
     556      0     11755   39.6000      A16        C
     571      0     11769   51.4792     C101        S
     587      1     13567   79.2000      B41        C
     591      0     36947   78.2667      D20        C
     599      0  PC 17485   56.9292      A20        C
     621      0     11753   52.5542      D19        S
     630      0     27042   30.0000      A23        S
     645      0  PC 17572   76.7292      D33        C
     647      0     13213   35.5000      A26        C
     660      0  PC 17611  133.6500  Unknown        S
     707      0  PC 17476   26.2875      E24        S
     712      0     19996   52.0000     C126        S
     765      0     13502   77.9583      D11        S
     779      1     24160  211.3375       B3        S
```

```
#Nivass C R - 241801193
#EX-2
#step 12

result = df[(df['Sex']=="male") &
            (df['Fare']>100)]

print(result[['Name','Fare']])
```

```
                                          Name      Fare
27                   Fortune, Mr. Charles Alexander  263.0000
118                      Baxter, Mr. Quigg Edmond  247.5208
305                Allison, Master. Hudson Trevor  151.5500
332                  Graham, Mr. George Edward  153.4625
373                        Ringhini, Mr. Sante  135.6333
377                    Widener, Mr. Harry Elkins  211.5000
390                  Carter, Mr. William Ernest  120.0000
438                       Fortune, Mr. Mark  263.0000
505  Penasco y Castellana, Mr. Victor de Satode  108.9000
527                          Farthing, Mr. John  221.7792
544                   Douglas, Mr. Walter Donald  106.4250
550                 Thayer, Mr. John Borland Jr  110.8833
557                     Robbins, Mr. Victor  227.5250
659                    Newell, Mr. Arthur Webster  113.2750
660                Frauenthal, Dr. Henry William  133.6500
679           Cardeza, Mr. Thomas Drake Martinez  512.3292
698                    Thayer, Mr. John Borland  110.8833
737                     Lesurer, Mr. Gustave J  512.3292
802           Carter, Master. William Thornton II  120.0000
```

```
#Nivass C R - 241801193
#EX-2
#step 13

result = df[(df['Embarked']=="C") &
            (df['Pclass']==2)]

print(result)
```

```
     PassengerId  Survived  Pclass  \
9             10         1       2
43            44         1       2
122          123         0       2
135          136         0       2
181          182         0       2
292          293         0       2
308          309         0       2
361          362         0       2
389          390         1       2
473          474         1       2
547          548         1       2
608          609         1       2
685          686         0       2
817          818         0       2
827          828         1       2
866          867         1       2
874          875         1       2

                                          Name     Sex        Age  \
9                Nasser, Mrs. Nicholas (Adele Achem)  female  14.000000
43             Laroche, Miss. Simonne Marie Anne Andree  female   3.000000
122                              Nasser, Mr. Nicholas    male  32.500000
135                              Richard, Mr. Emile    male  23.000000
181                                 Pernot, Mr. Rene    male  29.699118
292                           Levy, Mr. Rene Jacques    male  36.000000
308                            Abelson, Mr. Samuel    male  30.000000
361                         del Carlo, Mr. Sebastiano    male  29.000000
389                             Lehmann, Miss. Bertha  female  17.000000
473          Jerwan, Mrs. Amin S (Marie Marthe Thuillard)  female  23.000000
547                          Padro y Manent, Mr. Julian    male  29.699118
608  Laroche, Mrs. Joseph (Juliette Marie Louise La...  female  22.000000
685             Laroche, Mr. Joseph Philippe Lemercier    male  25.000000
817                                 Mallet, Mr. Albert    male  31.000000
827                           Mallet, Master. Andre     male   1.000000
866                       Duran y More, Miss. Asuncion  female  27.000000
874              Abelson, Mrs. Samuel (Hannah Wizosky)  female  28.000000

     SibSp  Parch      Ticket     Fare    Cabin Embarked
9        1      0      237736  30.0708  Unknown        C
43       1      2  SC/Paris 2123  41.5792  Unknown        C
122      1      0      237736  30.0708  Unknown        C
135      0      0  SC/PARIS 2133  15.0458  Unknown        C
181      0      0  SC/PARIS 2131  15.0500  Unknown        C
```

```
#Nivass C R - 241801193
#EX-2
#step 14

result = df[df['SibSp']>2]

print(result)
```

```
     686                             Panula, Mr. Jaako Arnold    male  14.000000
     726          Renouf, Mrs. Peter Henry (Lillian Jefferys)  female  30.000000
     787                           Rice, Master. George Hugh    male   8.000000
     792                        Sage, Miss. Stella Anna  female  29.699118
     813                  Andersson, Miss. Ebba Iris Alfrida  female   6.000000
     819                          Skoog, Master. Karl Thorsten    male  10.000000
     824                         Panula, Master. Urho Abraham    male   2.000000
     846                           Sage, Mr. Douglas Bullen    male  29.699118
     850               Andersson, Master. Sigvard Harald Elias    male   4.000000
     863                     Sage, Miss. Dorothy Edith "Dolly"  female  29.699118

          SibSp  Parch     Ticket      Fare        Cabin Embarked
     7        3      1     349909   21.0750      Unknown        S
     16       4      1     382652   29.1250      Unknown        Q
     24       3      1     349909   21.0750      Unknown        S
     27       3      2      19950  263.0000  C23 C25 C27        S
     50       4      1    3101295   39.6875      Unknown        S
     59       5      2    CA 2144   46.9000      Unknown        S
     63       3      2     347088   27.9000      Unknown        S
     68       4      2    3101281    7.9250      Unknown        S
     71       5      2    CA 2144   46.9000      Unknown        S
     85       3      0    3101278   15.8500      Unknown        S
     88       3      2      19950  263.0000  C23 C25 C27        S
     119      4      2     347082   31.2750      Unknown        S
     159      8      2    CA. 2343   69.5500      Unknown        S
     164      4      1    3101295   39.6875      Unknown        S
     171      4      1     382652   29.1250      Unknown        Q
     176      3      1       4133   25.4667      Unknown        S
     180      8      2    CA. 2343   69.5500      Unknown        S
     182      4      2     347077   31.3875      Unknown        S
     201      8      2    CA. 2343   69.5500      Unknown        S
     229      3      1       4133   25.4667      Unknown        S
     233      4      2     347077   31.3875      Unknown        S
     261      4      2     347077   31.3875      Unknown        S
     266      4      1    3101295   39.6875      Unknown        S
     278      4      1     382652   29.1250      Unknown        Q
     324      8      2    CA. 2343   69.5500      Unknown        S
     341      3      2      19950  263.0000  C23 C25 C27        S
     374      3      1     349909   21.0750      Unknown        S
     386      5      2    CA 2144   46.9000      Unknown        S
     409      3      1       4133   25.4667      Unknown        S
     480      5      2    CA 2144   46.9000      Unknown        S
     485      3      1       4133   25.4667      Unknown        S
     541      4      2     347082   31.2750      Unknown        S
     542      4      2     347082   31.2750      Unknown        S
     634      3      2     347088   27.9000      Unknown        S
     642      3      2     347088   27.9000      Unknown        S
     683      5      2    CA 2144   46.9000      Unknown        S
     686      4      1    3101295   39.6875      Unknown        S
     726      3      0      31027   21.0000      Unknown        S
     787      4      1     382652   29.1250      Unknown        Q
     792      8      2    CA. 2343   69.5500      Unknown        S
     813      4      2     347082   31.2750      Unknown        S
     819      3      2     347088   27.9000      Unknown        S
     824      4      1    3101295   39.6875      Unknown        S
     846      8      2    CA. 2343   69.5500      Unknown        S
     850      4      2     347082   31.2750      Unknown        S
     863      8      2    CA. 2343   69.5500      Unknown        S
```

```
#Nivass C R - 241801193
#EX-2
#step 15

result = df[(df['Survived']==0) &
            (df['SibSp']==0) &
            (df['Parch']==0)]

print(result)
```

```
     PassengerId  Survived  Pclass                                Name  \
4              5         0       3             Allen, Mr. William Henry
5              6         0       3                     Moran, Mr. James
6              7         0       1              McCarthy, Mr. Timothy J
12            13         0       3       Saundercock, Mr. William Henry
14            15         0       3  Vestrom, Miss. Hulda Amanda Adolfina
..           ...       ...     ...                                 ...
882          883         0       3          Dahlberg, Miss. Gerda Ulrika
883          884         0       2         Banfield, Mr. Frederick James
884          885         0       3                Sutehall, Mr. Henry Jr
886          887         0       2                  Montvila, Rev. Juozas
890          891         0       3                   Dooley, Mr. Patrick

        Sex        Age  SibSp  Parch            Ticket     Fare    Cabin  \
4      male  35.000000      0      0            373450   8.0500  Unknown
5      male  29.699118      0      0            330877   8.4583  Unknown
6      male  54.000000      0      0             17463  51.8625      E46
12     male  20.000000      0      0         A/5. 2151   8.0500  Unknown
14   female  14.000000      0      0            350406   7.8542  Unknown
..      ...        ...    ...    ...               ...      ...      ...
882  female  22.000000      0      0              7552  10.5167  Unknown
883    male  28.000000      0      0  C.A./SOTON 34068  10.5000  Unknown
884    male  25.000000      0      0   SOTON/OQ 392076   7.0500  Unknown
886    male  27.000000      0      0            211536  13.0000  Unknown
890    male  32.000000      0      0            370376   7.7500  Unknown

    Embarked
4          S
5          Q
6          S
12         S
14         S
..       ...
882        S
883        S
884        S
886        S
890        Q

[374 rows x 12 columns]
```

```
#Nivass C R - 241801193
#EX-2
#step 16

result = df[df['Survived']==1]

result = result.sort_values(by='Age', ascending=False)

print(result.head(5))
```

```
     PassengerId  Survived  Pclass                                  Name  \
630          631         1       1       Barkworth, Mr. Algernon Henry Wilson
275          276         1       1         Andrews, Miss. Kornelia Theodosia
483          484         1       3                     Turkula, Mrs. (Hedwig)
570          571         1       2                      Harris, Mr. George
829          830         1       1  Stone, Mrs. George Nelson (Martha Evelyn)

        Sex   Age  SibSp  Parch       Ticket     Fare    Cabin Embarked
630    male  80.0      0      0        27042  30.0000      A23        S
275  female  63.0      1      0        13502  77.9583       D7        S
483  female  63.0      0      0         4134   9.5875  Unknown        S
570    male  62.0      0      0  S.W./PP 752  10.5000  Unknown        S
829  female  62.0      0      0       113572  80.0000      B28        S
```

```
#Nivass C R - 241801193
#EX-2
#step 17

result = df[df['Fare']==0]

print(result[['Name','Fare']])
```

```
                                 Name  Fare
179              Leonard, Mr. Lionel   0.0
263            Harrison, Mr. William   0.0
271      Tornquist, Mr. William Henry   0.0
277       Parkes, Mr. Francis "Frank"   0.0
302   Johnson, Mr. William Cahoone Jr   0.0
413    Cunningham, Mr. Alfred Fleming   0.0
466            Campbell, Mr. William   0.0
481  Frost, Mr. Anthony Wood "Archie"   0.0
597                Johnson, Mr. Alfred   0.0
633     Parr, Mr. William Henry Marsh   0.0
674        Watson, Mr. Ennis Hastings   0.0
732              Knight, Mr. Robert J   0.0
806            Andrews, Mr. Thomas Jr   0.0
815                 Fry, Mr. Richard   0.0
822  Reuchlin, Jonkheer. John George   0.0
```

```
#Nivass C R - 241801193
#EX-2
#step 18

# Select Features and Target
X = df.drop('Survived', axis=1)
y = df['Survived']

# Convert Categorical Columns
X = pd.get_dummies(X)

# Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Check Shape
print(X_train.shape)
print(X_test.shape)
```

```
(712, 1731)
(179, 1731)
```

Ex3a: Model Planning and Building – Classification

Learning Objective: The objective of this experiment is to enable students to plan, build, and evaluate basic machine learning models using real-world datasets.

Problem Statement:

A telecommunications company has noticed that a growing number of customers are discontinuing its services, resulting in reduced revenue and increased customer acquisition costs. After completing data preprocessing and exploratory analysis, the company now aims to develop predictive models that can identify customers who are likely to churn. Early prediction of customer churn will enable the company to implement targeted retention strategies, improve customer satisfaction, and reduce business losses.

As a Data Scientist, your task is to build and compare multiple classification models to predict whether a customer will churn. You will train different machine learning algorithms, evaluate their performance using appropriate classification metrics, and identify the most effective model for customer churn prediction. Finally, interpret the important factors influencing customer churn and communicate your findings through suitable visualizations and performance comparisons.

Tasks to Perform: 1. Use CustomerChurn_Preprocessed.csv file which you created in Ex2.

2. Split the cleaned dataset into Training (80%) and Testing (20%) datasets.

Model Building

3. Train a Logistic Regression classifier and predict the customer churn on the test dataset.

4. Train a Decision Tree Classifier and predict the customer churn on the test dataset.

5. Train a Random Forest Classifier and predict the customer churn on the test dataset. Model Evaluation 6. Evaluate each model using Confusion Matrix, Accuracy, Precision, Recall, F1-Score.

7. Compare the performance of all three models using a single bar chart showing the evaluation metrics. 8. Identify the best-performing model based on the evaluation results.

Business Recommendation

9. Based on the best-performing model, provide three recommendations that the telecommunications company can implement to reduce customer churn.

```python
# Name: Nivass C R
# Roll No: 241801193
#Exp-3a
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score, classification_report
)
```

```python
# Nivass C R
# Roll No: 241801193
from google.colab import files
uploaded = files.upload()
filename = next(iter(uploaded))
df = pd.read_csv(filename)
print("Dataset Shape:", df.shape)
display(df.head())
df.info()
```

```
Choose files  WA_Fn-Use...r-Churn.csv
WA_Fn-UseC_-Telco-Customer-Churn.csv(text/csv) - 977501 bytes, last modified: 14/10/2026 - 100% done
Saving WA_Fn-UseC_-Telco-Customer-Churn.csv to WA_Fn-UseC_-Telco-Customer-Churn (2).csv
Dataset Shape: (7043, 21)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtect
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	

```python
# Name: Nivass C R
# Roll No: 241801193
df = df.drop(columns=["CustomerID"], errors="ignore")
if df["Churn"].dtype == "object":
    df["Churn"] = df["Churn"].astype(str).str.strip().map(
        {"Yes": 1, "No": 0}
    )
df = df.dropna(subset=["Churn"])
X = pd.get_dummies(df.drop(columns=["Churn"]), drop_first=True)
y = df["Churn"].astype(int)
X = X.replace([np.inf, -np.inf], np.nan)
X = X.fillna(X.median())
print("Features:", X.shape)
print("Target Distribution:")
print(y.value_counts())
```

```
Features: (7043, 13601)
Target Distribution:
Churn
0    5174
1    1869
Name: count, dtype: int64
```

```python
# Name: Nivass C R
# Roll No: 241801193
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (5634, 13601)
Testing Data: (1409, 13601)
```

```python
# Name: Nivass C R
# Roll No: 241801193
lr = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=1000)
)

lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
print("Logistic Regression")
print(confusion_matrix(y_test, y_pred_lr))
print(classification_report(y_test, y_pred_lr))
```

```
Logistic Regression
[[1011   24]
 [ 309   65]]
              precision    recall  f1-score   support

           0       0.77      0.98      0.86      1035
           1       0.73      0.17      0.28       374

    accuracy                           0.76      1409
   macro avg       0.75      0.58      0.57      1409
weighted avg       0.76      0.76      0.71      1409
```

```python
# Name: Nivass C R
# Roll No: 241801193
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)
print("Decision Tree")
print(confusion_matrix(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))
```

```
Decision Tree
[[890 145]
 [185 189]]
              precision    recall  f1-score   support

           0       0.83      0.86      0.84      1035
           1       0.57      0.51      0.53       374

    accuracy                           0.77      1409
   macro avg       0.70      0.68      0.69      1409
weighted avg       0.76      0.77      0.76      1409
```

```python
# Name: Nivass C R
# Roll No: 241801193
rf = RandomForestClassifier(
    n_estimators=100, random_state=42
)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
print("Random Forest")
print(confusion_matrix(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
```

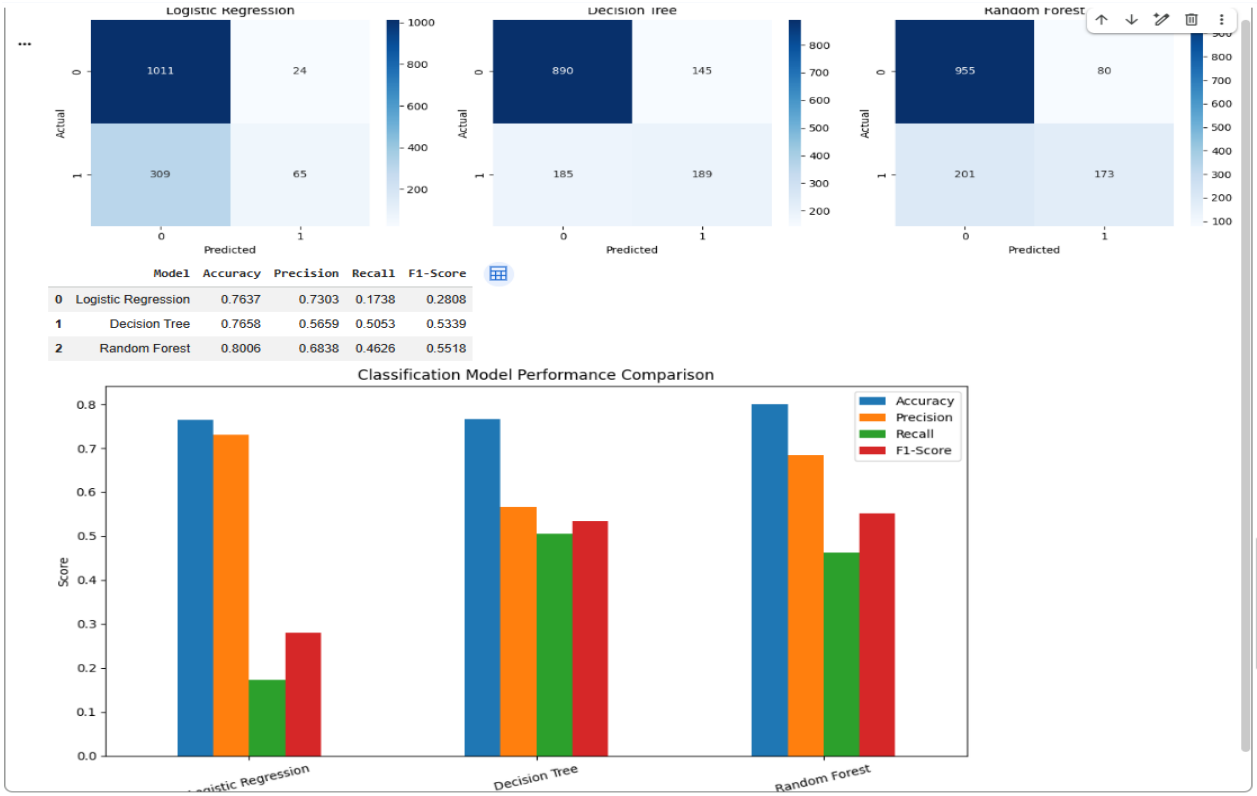
```
Random Forest
[[955  80]
 [201 173]]
              precision    recall  f1-score   support

           0       0.83      0.92      0.87      1035
           1       0.68      0.46      0.55       374

    accuracy                           0.80      1409
   macro avg       0.75      0.69      0.71      1409
weighted avg       0.79      0.80      0.79      1409
```

```python
# Name: Nivass C R
# Roll No: 241801193
models = {
    "Logistic Regression": (lr, y_pred_lr),
    "Decision Tree": (dt, y_pred_dt),
    "Random Forest": (rf, y_pred_rf)
}
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, (name, (model, pred)) in zip(axes, models.items()):
    cm = confusion_matrix(y_test, pred)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax)
    ax.set_title(name)
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")

plt.tight_layout()
plt.show()
results = []
for name, (model, pred) in models.items():
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred, zero_division=0),
        "Recall": recall_score(y_test, pred, zero_division=0),
        "F1-Score": f1_score(y_test, pred, zero_division=0)
    })
results_df = pd.DataFrame(results)
display(results_df.round(4))
results_df.set_index("Model").plot(
    kind="bar", figsize=(10, 6)
)
plt.title("Classification Model Performance Comparison")
plt.ylabel("Score")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```



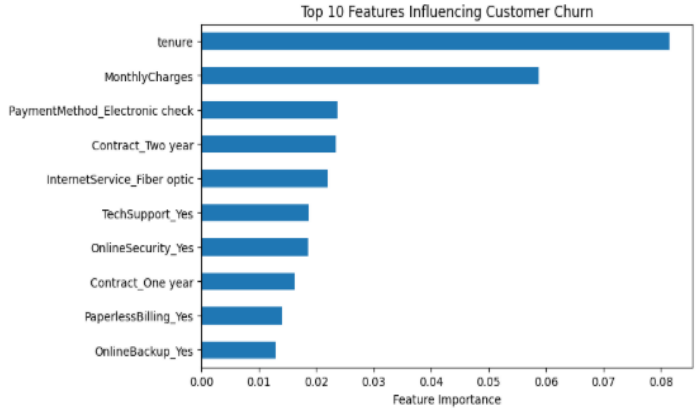
	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.7637	0.7303	0.1738	0.2808
1	Decision Tree	0.7658	0.5659	0.5053	0.5339
2	Random Forest	0.8006	0.6838	0.4626	0.5518


```python
# Name: Nivass C R
# Roll No: 241801193
best_model = results_df.loc[
    results_df["F1-Score"].idxmax(), "Model"
]
print("Best Model:", best_model)
if best_model == "Random Forest":
    importance = rf.feature_importances_
elif best_model == "Decision Tree":
    importance = dt.feature_importances_
else:
    importance = None
if importance is not None:
    feature_importance = pd.Series(
        importance, index=X.columns
    ).sort_values(ascending=False)

    feature_importance.head(10).sort_values().plot(
        kind="barh", figsize=(8, 5)
    )
    plt.title("Top 10 Features Influencing Customer Churn")
    plt.xlabel("Feature Importance")
    plt.show()

print("\nBusiness Recommendations:")
print("1. Identify customers with a high risk of churn.")
print("2. Offer suitable plans and retention incentives.")
print("3. Improve customer support and customer satisfaction.")
```

```
Best Model: Random Forest
```




```
Business Recommendations:
1. Identify customers with a high risk of churn.
2. Offer suitable plans and retention incentives.
3. Improve customer support and customer satisfaction.
```

Ex3b: Model Planning and Building - Regression

Learning Objective: The objective of this experiment is to enable students to plan, build, and evaluate regression models using a real-world dataset.

Problem Statement: A marketing agency wants to help a retail company optimize its advertising budget. The company invests in three advertising channels—TV, Radio, and Newspaper—and wants to understand how these advertising expenditures influence product sales. By developing predictive models, the company aims to estimate future sales based on advertising investments and identify the most suitable regression model for accurate sales prediction.

As a Data Scientist, your task is to build and compare multiple regression models using the Advertising dataset. Evaluate the performance of each model using appropriate regression metrics and communicate the findings through visualizations to support business decision-making.

Tasks to Perform:

1. Download the Advertising Dataset from Kaggle and load it into a Pandas DataFrame.

2. Explore the dataset by: Displaying the first and last five records, Checking the dataset shape and data types, Identifying missing values and duplicate records, Displaying summary statistics.

3. Select: TV, Radio, Newspaper as Input Features (X), Sales as Target Variable (y).

Perform Exploratory Data Analysis:

4. Generate a Pair Plot for the selected features

. 5. Plot a Correlation Heatmap.

6. Identify the feature most strongly correlated with Sales.

Model Planning and Model Building

7. Split the dataset into Training (80%) and Testing (20%) dataset. 8

. Build a Linear Regression model and predict Sales for the test dataset.

9. Build a Polynomial Regression model (Degree = 2) and predict Sales for the test dataset.

10. Build a Support Vector Regression (SVR) model with the RBF kernel and predict Sales for the test dataset.

Model Evaluation

11. Evaluate all three models using: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R^2 Score

12. Compare the performance of all models using a single grouped bar chart.

13. Plot Actual vs Predicted Sales for the best-performing model.

Model Interpretation

14. Identify the model with the lowest prediction error.

15. Suggest three recommendations to help the company optimize its advertising budget based on the model results.

```
#Experiment-3b
#Step-1
#Nivass C R [241801193]

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.cluster import KMeans
```

```
#Experiment-3
#Step-2
#Nivass C R [241801193]

from google.colab import files

uploaded = files.upload()

df = pd.read_csv("Advertising.csv")

print(df.head())
```

```
Choose files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Advertising.csv to Advertising (1).csv
      TV  Radio  Newspaper  Sales
0  230.1   37.8       69.2   22.1
1   44.5   39.3       45.1   10.4
2   17.2   45.9       69.3   12.0
3  151.5   41.3       58.5   16.5
4  180.8   10.8       58.4   17.9
```

```
#Experiment-3
#Step-3
#Nivass C R [241801193]

# Display first 5 rows
print(df.head())

# Display information about dataset
print(df.info())

# Display statistical information
print(df.describe())

# Check missing values
print(df.isnull().sum())

# Check data types
print(df.dtypes)
```

```
      TV  Radio  Newspaper  Sales
0  230.1   37.8       69.2   22.1
1   44.5   39.3       45.1   10.4
2   17.2   45.9       69.3   12.0
3  151.5   41.3       58.5   16.5
4  180.8   10.8       58.4   17.9
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   TV         200 non-null    float64
 1   Radio      200 non-null    float64
 2   Newspaper  200 non-null    float64
 3   Sales      200 non-null    float64
dtypes: float64(4)
memory usage: 6.4 KB
None
               TV       Radio   Newspaper       Sales
count  200.000000  200.000000  200.000000  200.000000
mean   147.042500   23.264000   30.554000   15.130500
std     85.854236   14.846809   21.778621    5.283892
min      0.700000    0.000000    0.300000    1.600000
25%     74.375000    9.975000   12.750000   11.000000
50%    149.750000   22.900000   25.750000   16.000000
75%    218.825000   36.525000   45.100000   19.050000
max    296.400000   49.600000  114.000000   27.000000
TV           0
Radio        0
Newspaper    0
Sales        0
dtype: int64
```

```python
#Experiment-3
#Step-4
#Nivass C R [241801193]

# Select input features
X = df[['TV', 'Radio', 'Newspaper']]

# Select target/output
y = df['Sales']

print("Features:")
print(X.head())

print("\nTarget:")
print(y.head())
```

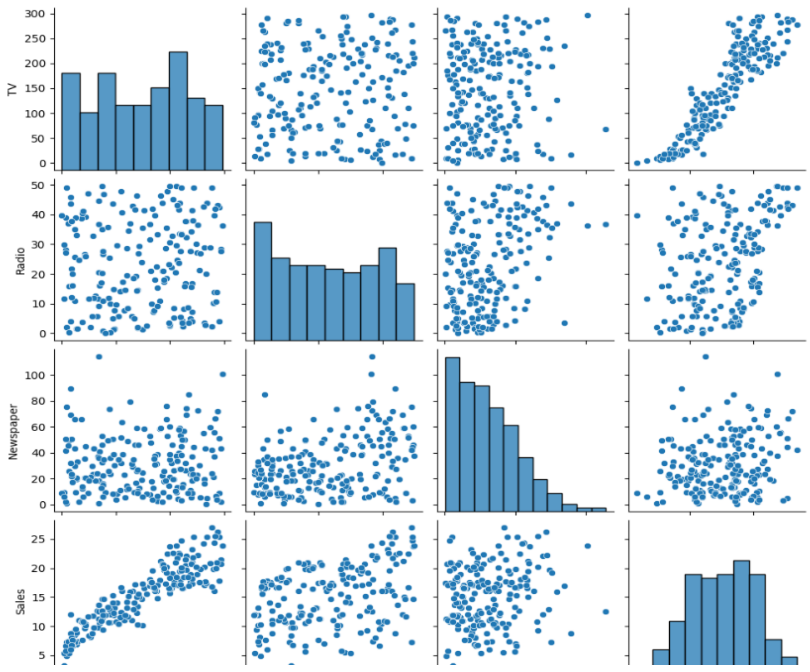
```
Features:
      TV  Radio  Newspaper
0  230.1   37.8       69.2
1   44.5   39.3       45.1
2   17.2   45.9       69.3
3  151.5   41.3       58.5
4  180.8   10.8       58.4

Target:
0    22.1
1    10.4
2    12.0
3    16.5
4    17.9
Name: Sales, dtype: float64
```

```python
#Experiment-3
#Step-5
#Nivass C R [241801193]

sns.pairplot(df)

plt.show()
```




```python
#Experiment-3
#Step-6
#Nivass C R [241801193]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (160, 3)
Testing data: (40, 3)
```

```python
#Experiment-3
#Step-7
#Nivass C R [241801193]

# Create Linear Regression model
lr = LinearRegression()

# Train the model
lr.fit(X_train, y_train)

# Predict Sales on test data
y_pred_lr = lr.predict(X_test)

print("Linear Regression Predictions:")
print(y_pred_lr)
```

```
Linear Regression Predictions:
[17.0347724  20.40974033 23.72398873  9.27278518 21.68271879 12.56940161
 21.08119452  8.69035045 17.23701254 16.66657475  8.92396497  8.4817344
 18.2075123   8.06750728 12.64550975 14.93162809  8.12814594 17.89876565
 11.00880637 20.47832788 20.80631846 12.59883297 10.9051829  22.38854775
  9.41796094  7.92506736 20.83908497 13.81520938 10.77080925  7.92682509
 15.95947357 10.63490851 20.80292008 10.43434164 21.5784752  21.18364487
 12.12821771 22.80953262 12.60992766  6.46441252]
```

```python
#Experiment-3
#Step-8
#Nivass C R [241801193]

# Create Polynomial Regression pipeline
poly_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)

# Train the model
poly_model.fit(X_train, y_train)

# Predict Sales on test data
y_pred_poly = poly_model.predict(X_test)

print("Polynomial Regression Predictions:")
print(y_pred_poly)
```

```
Polynomial Regression Predictions:
[17.562438   21.96285664 22.07580481  7.34916293 23.23684435 12.37109504
 22.12872506  8.76904874 17.01492635 16.91480608  7.86402513  9.1023732
 17.43691412  5.99987179 13.70960498 15.38247085  6.28283192 18.02070399
 10.95120455 20.20193781 20.93683157 13.01896785  9.65505427 22.48602888
 10.0006458   8.88804873 21.83258724 14.37821146 11.24748724  6.08610266
 16.1977029  10.4879825  22.56616524  9.20046174 19.54142042 21.31849501
 11.27647626 21.73705841 13.18303888  7.12553149]
```

```
#Experiment-3
#Step-9
#Nivass C R [241801193]

# Create SVR model with RBF kernel
svr = SVR(kernel='rbf')

# Train the model
svr.fit(X_train, y_train)

# Predict Sales on test data
y_pred_svr = svr.predict(X_test)

print("SVR Predictions:")
print(y_pred_svr)
```

```
SVR Predictions:
[17.15330196 19.49875585 20.96245814  8.60209215 19.88553201 12.03803929
 19.98237498  9.22752786 17.73379117 17.08961377  8.99691527  9.3292374
 18.62744348  8.14145921 14.08177689 16.00015525  8.04682867 17.97920438
 11.12951222 19.55761255 20.0541767  12.95980908  9.9170165  20.86239638
 10.43376345  9.07422216 19.89744797 14.71765498 11.47650759  8.24606316
 16.91068571 10.75656165 19.72411632 10.2142965  20.15216829 19.79413534
 10.88647539 20.96881165  13.5160367   7.85022468]
```

```
#Experiment-3
#Step-10
#Nivass C R [241801193]

# Store predictions of all models
models = {
    "Linear Regression": y_pred_lr,
    "Polynomial Regression": y_pred_poly,
    "SVR": y_pred_svr
}

# Evaluate each model
for name, pred in models.items():

    mae = mean_absolute_error(y_test, pred)
    mse = mean_squared_error(y_test, pred)
    rmse = np.sqrt(mse)

    print("\n", name)
    print("MAE :", mae)
    print("MSE :", mse)
    print("RMSE:", rmse)
```

```
 Linear Regression
MAE : 1.2748262109549338
MSE : 2.9077569102710896
RMSE: 1.7052146229349223

 Polynomial Regression
MAE : 0.9033713382792442
MSE : 1.44254263690159
RMSE: 1.2010589647896517

 SVR
MAE : 1.4836223881587627
MSE : 3.499087197255254
RMSE: 1.8705847206836834
```

```
#Experiment-3
#Step-11
#Nivass C R [241801193]
# Calculate evaluation metrics
mae_values = []
mse_values = []
rmse_values = []
model_names = []

for name, pred in models.items():

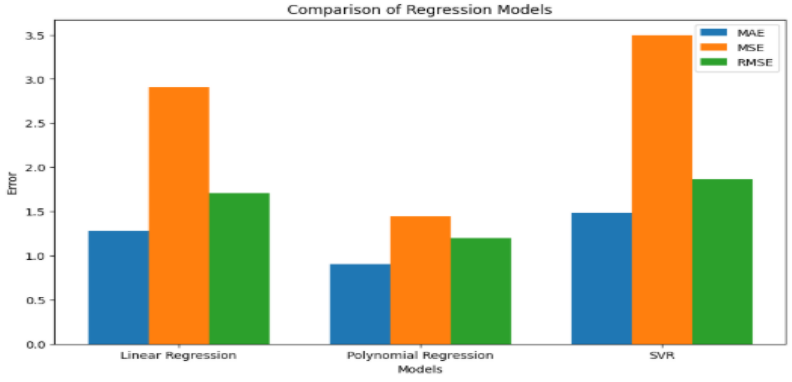
    mae = mean_absolute_error(y_test, pred)
    mse = mean_squared_error(y_test, pred)
    rmse = np.sqrt(mse)

    model_names.append(name)
    mae_values.append(mae)
    mse_values.append(mse)
    rmse_values.append(rmse)

# Create a single bar graph
x = np.arange(len(model_names))
width = 0.25

plt.figure(figsize=(10, 6))
plt.bar(x - width, mae_values, width, label='MAE')
plt.bar(x, mse_values, width, label='MSE')
plt.bar(x + width, rmse_values, width, label='RMSE')

plt.xlabel("Models")
plt.ylabel("Error")
plt.title("Comparison of Regression Models")
plt.xticks(x, model_names)
plt.legend()
plt.show()
```



```
#Experiment-3
#Step-12
#Nivass C R [241801193]

# Create K-Means model
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

# Fit K-Means using feature data
kmeans.fit(X)

# Add cluster labels to DataFrame
df['Cluster'] = kmeans.labels_

print(df.head())
```

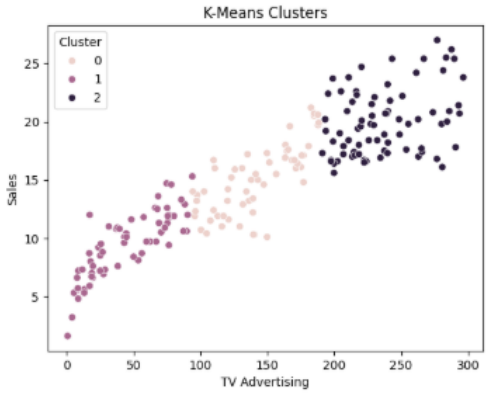
```
      TV  Radio  Newspaper  Sales  Cluster
0  230.1   37.8       69.2   22.1        2
1   44.5   39.3       45.1   10.4        1
2   17.2   45.9       69.3   12.0        1
3  151.5   41.3       58.5   16.5        0
4  180.8   10.8       58.4   17.9        0
```

```
#Experiment-3
#Step-13
#Nivass C R [241801193]

sns.scatterplot(
    data=df,
    x='TV',
    y='Sales',
    hue='Cluster'
)

plt.xlabel("TV Advertising")
plt.ylabel("Sales")
plt.title("K-Means Clusters")

plt.show()
```



Ex4: Information Retrieval using NLTK

Learning Objective: The objective of this experiment is to enable students to understand the text preprocessing pipeline and implement a basic information retrieval system using TF-IDF vectorization, Cosine Similarity, NLTK.

Problem Statement:

An e-commerce company receives thousands of customer reviews for its products every day. Customer support executives often need to retrieve reviews that are similar to a user's search query (e.g., "great product with fast shipping" or "poor quality") to quickly understand customer feedback and address common concerns.

As an NLP Engineer, your task is to preprocess customer review text, convert the reviews into numerical feature vectors using the TF-IDF (Term Frequency–Inverse Document Frequency) technique, and develop a simple information retrieval system that returns the most relevant customer reviews for a given search query using Cosine Similarity.

Tasks to Perform:

1. Use Amazon Fine Food Reviews Dataset from Kaggle and load it into a Pandas DataFrame.

2. Install and import the required Python libraries:

3. Import the required NLTK resources

Clean the dataset

4. Select the Text column containing customer reviews, Remove missing or null values, Remove duplicate reviews (if any), Retain only the first 10,000 reviews for faster processing. Text Preprocessing

5. Create a preprocessing function to perform the following operations on each review: Convert text to lowercase, Remove punctuation, numbers, and special characters, Tokenize the text into words, Remove English stopwords, Join the cleaned tokens into a single string.

6. Apply the preprocessing function to all reviews and store the cleaned reviews in a new column named Cleaned_Text.

7. Display the original and cleaned versions of any five reviews.

8. Convert the cleaned review text into TF-IDF feature vectors. Display: Shape of the TF-IDF matrix, Total number of unique terms (vocabulary size).

Information Retrieval

10. Implement a document retrieval function that: Accepts a user query, Applies the same preprocessing steps used for the reviews, Converts the query into a TF-IDF vector, Computes Cosine Similarity between the query and all review vectors, Retrieves the Top 5 most similar reviews, Display Similarity score, Original review, Cleaned review.

Query Evaluation

11. Test the retrieval system using the following sample queries: "great product with fast shipping", "disappointed", "excellent taste", "poor packaging"

12. Compare the retrieved reviews for different queries and comment on their relevance.

```python
#Experiment-4
#Step-1
#Nivass C R

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.cluster import KMeans
```

```python
#Experiment-4
#Step-2
#Nivass C R

from google.colab import files
uploaded = files.upload()
df = pd.read_csv("house_prices.csv")

print("First 5 rows:")
print(df.head())
```

```
Choose files  No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving house_prices.xlsx to house_prices.xlsx
First 5 rows:
      price  area  bedrooms  bathrooms  stories mainroad guestroom basement  \
0  13300000  7420         4          2        3      yes        no       no
1  12250000  8960         4          4        4      yes        no       no
2  12250000  9960         3          2        2      yes        no      yes
3  12215000  7500         4          2        2      yes        no      yes
4  11410000  7420         4          1        2      yes       yes      yes

  hotwaterheating airconditioning  parking prefarea furnishingstatus
0              no             yes        2      yes        furnished
1              no             yes        3       no        furnished
2              no              no        2      yes   semi-furnished
3              no             yes        3      yes        furnished
4              no             yes        2       no        furnished
```

```python
#Experiment-4
#Step-3
#Nivass C R

print("\nDataset Information:")
print(df.info())
print("\nStatistical Description:")
print(df.describe())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   price             545 non-null    int64
 1   area              545 non-null    int64
 2   bedrooms          545 non-null    int64
 3   bathrooms         545 non-null    int64
 4   stories           545 non-null    int64
 5   mainroad          545 non-null    object
 6   guestroom         545 non-null    object
 7   basement          545 non-null    object
 8   hotwaterheating   545 non-null    object
 9   airconditioning   545 non-null    object
 10  parking           545 non-null    int64
 11  prefarea          545 non-null    object
 12  furnishingstatus  545 non-null    object
dtypes: int64(6), object(7)
memory usage: 55.5+ KB
None

Statistical Description:
              price          area    bedrooms   bathrooms     stories  \
count  5.450000e+02    545.000000  545.000000  545.000000  545.000000
mean   4.766729e+06   5150.541284    2.965138    1.286239    1.805505
std    1.870440e+06   2170.141023    0.738064    0.502470    0.867492
min    1.750000e+06   1650.000000    1.000000    1.000000    1.000000
25%    3.430000e+06   3600.000000    2.000000    1.000000    1.000000
50%    4.340000e+06   4600.000000    3.000000    1.000000    2.000000
75%    5.740000e+06   6360.000000    3.000000    2.000000    2.000000
max    1.330000e+07  16200.000000    6.000000    4.000000    4.000000

          parking
count  545.000000
mean     0.693578
std      0.861586
min      0.000000
```

```
#Experiment-4
#Step-4
#Nivass C R

print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
price             0
area              0
bedrooms          0
bathrooms         0
stories           0
mainroad          0
guestroom         0
basement          0
hotwaterheating   0
airconditioning   0
parking           0
prefarea          0
furnishingstatus  0
dtype: int64
```

```
#Experiment-4
#Step-5
#Nivass C R

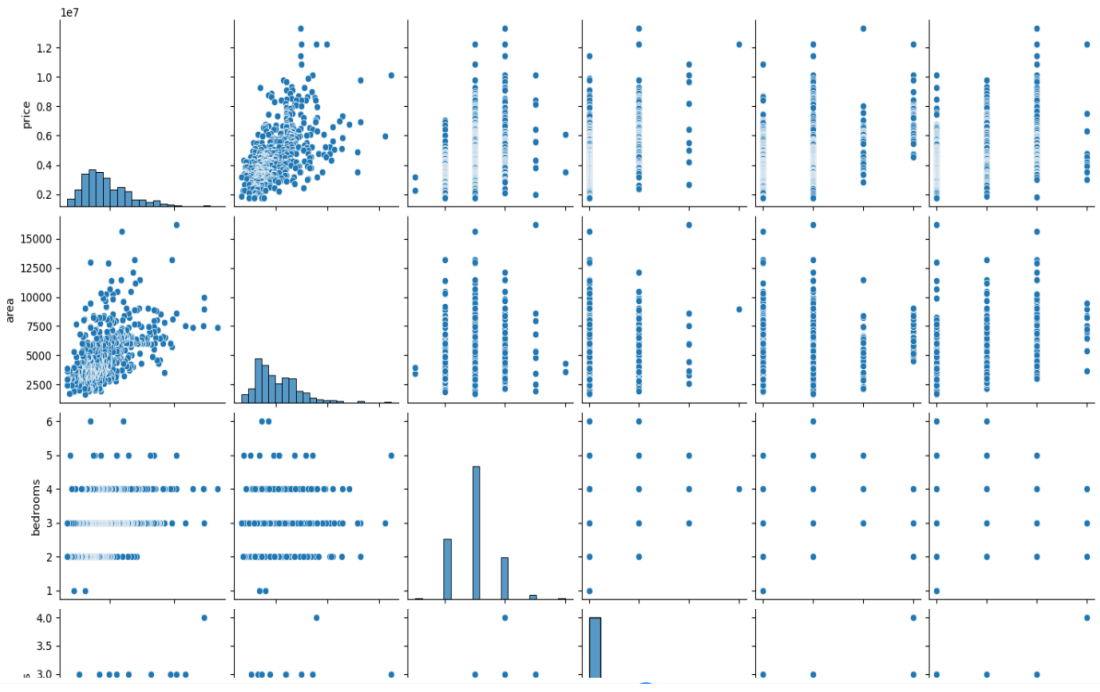
X = df[['area', 'bedrooms', 'bathrooms', 'stories', 'parking']]
y = df['price']
print("\nFeatures:")
print(X.head())
print("\nTarget:")
print(y.head())
```

```
Features:
   area  bedrooms  bathrooms  stories  parking
0  7420         4          2        3        2
1  8960         4          4        4        3
2  9960         3          2        2        2
3  7500         4          2        2        3
4  7420         4          1        2        2

Target:
0    13300000
1    12250000
2    12250000
3    12215000
4    11410000
Name: price, dtype: int64
```

```
#Experiment-4
#Step-6
#Nivass C R

sns.pairplot(df)
plt.show()
```



```
#Experiment-4
#Step-7
#Nivass C R

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (436, 5)
Testing data: (109, 5)
```

```
#Experiment-4
#Step-8
#Nivass C R

linear_model = LinearRegression()

linear_model.fit(X_train, y_train)

y_pred_linear = linear_model.predict(X_test)

print("Linear Regression Predictions:")
print(y_pred_linear[:10])
```

```
Linear Regression Predictions:
[6178627.50326817 6370140.85865473 3283148.1570249  4226007.94816032
 3409685.55116741 4262158.3499693  5493440.53152357 5559897.74293016
 3373715.42245305 3020513.18649699]
```

```
#Experiment-4
#Step-9
#Nivass C R

polynomial_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)

polynomial_model.fit(X_train, y_train)

y_pred_poly = polynomial_model.predict(X_test)

print("Polynomial Regression Predictions:")
print(y_pred_poly[:10])
```

```
Polynomial Regression Predictions:
[6505190.04806989 6264935.34475896 3379732.28086229 4313632.08029262
 3609277.61754304 4685697.11593135 5823994.6723154  5292998.03249803
 3206392.32827146 2914176.45201578]
```

```
#Experiment-4
#Step-10
#Nivass C R

from sklearn.preprocessing import StandardScaler

svr_model = make_pipeline(
    StandardScaler(),
    SVR(kernel='rbf')
)

svr_model.fit(X_train, y_train)

y_pred_svr = svr_model.predict(X_test)

print("SVR Predictions:")
print(y_pred_svr[:10])
```

```
SVR Predictions:
[4291030.24435396 4291024.79071864 4290935.38648078 4290962.44930117
 4290944.98176684 4290977.98903694 4291018.39490216 4291014.88261073
 4290945.35746905 4290940.76642675]
```

```python
#Experiment-4
#Step-11
#Nivass C R

models = {
    "Linear Regression": y_pred_linear,
    "Polynomial Regression": y_pred_poly,
    "SVR": y_pred_svr
}

for name, prediction in models.items():

    mae = mean_absolute_error(y_test, prediction)
    mse = mean_squared_error(y_test, prediction)
    rmse = np.sqrt(mse)

    print("\nModel:", name)
    print("MAE :", mae)
    print("MSE :", mse)
    print("RMSE:", rmse)
```

```
Model: Linear Regression
MAE : 1127483.352323519
MSE : 2292721545725.3623
RMSE: 1514173.5520492233

Model: Polynomial Regression
MAE : 1111158.8115570166
MSE : 2326140712896.4272
RMSE: 1525169.0768227722

Model: SVR
MAE : 1763886.6428081566
MSE : 5567921966485.061
RMSE: 2359644.4576429436
```

```python
#Experiment-4
#Step-12
#Nivass C R

from sklearn.preprocessing import StandardScaler

cluster_data = df[['area', 'bedrooms', 'bathrooms', 'stories', 'parking']]

scaler = StandardScaler()
scaled_data = scaler.fit_transform(cluster_data)

kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

df['Cluster'] = kmeans.fit_predict(scaled_data)

print("\nCluster Labels:")
print(df[['area', 'bedrooms', 'bathrooms', 'stories', 'parking', 'Cluster']].head(20))
```

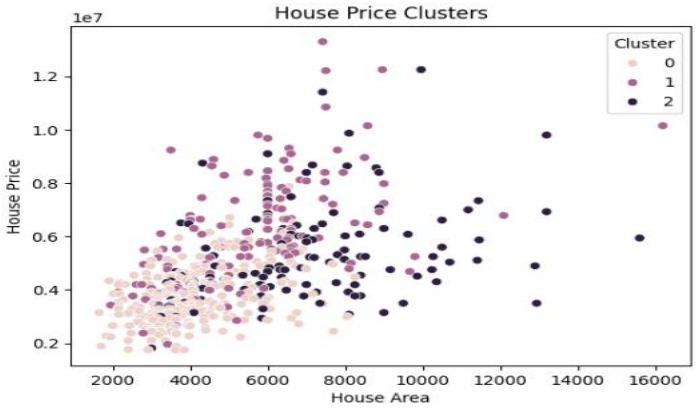
```
Cluster Labels:
     area  bedrooms  bathrooms  stories  parking  Cluster
0    7420         4          2        3        2        1
1    8960         4          4        4        3        1
2    9960         3          2        2        2        2
3    7500         4          2        2        3        1
4    7420         4          1        2        2        2
5    7500         3          3        1        2        1
6    8580         4          3        4        2        1
7   16200         5          3        2        0        1
8    8100         4          1        2        2        2
9    5750         3          2        4        1        1
10  13200         3          1        2        2        2
11   6000         4          3        2        2        1
12   6550         4          2        2        1        1
13   3500         4          2        2        2        1
14   7800         3          2        2        0        1
15   6000         4          1        2        2        2
16   6600         4          2        2        1        1
17   8500         3          2        4        2        1
18   4600         3          2        2        2        1
19   6420         3          2        2        1        1
```

```
#Experiment-4
#Step-13
#Nivass C R

sns.scatterplot(
    data=df,
    x='area',
    y='price',
    hue='Cluster'
)

plt.xlabel("House Area")
plt.ylabel("House Price")
plt.title("House Price Clusters")

plt.show()
```




```
#Experiment-4
#Step-14
#Nivass C R

print("\nFinal Dataset with Cluster Labels:")
print(df.head(20))
```

```
Final Dataset with Cluster Labels:
       price   area  bedrooms  bathrooms  stories mainroad guestroom basement  \
0   13300000   7420         4          2        3      yes        no       no
1   12250000   8960         4          4        4      yes        no       no
2   12250000   9960         3          2        2      yes        no      yes
3   12215000   7500         4          2        2      yes        no      yes
4   11410000   7420         4          1        2      yes       yes      yes
5   10850000   7500         3          3        1      yes        no      yes
6   10150000   8580         4          3        4      yes        no       no
7   10150000  16200         5          3        2      yes        no       no
8    9870000   8100         4          1        2      yes       yes      yes
9    9800000   5750         3          2        4      yes       yes       no
10   9800000  13200         3          1        2      yes        no      yes
11   9681000   6000         4          3        2      yes       yes      yes
12   9310000   6550         4          2        2      yes        no       no
13   9240000   3500         4          2        2      yes        no       no
14   9240000   7800         3          2        2      yes        no       no
15   9100000   6000         4          1        2      yes        no      yes
16   9100000   6600         4          2        2      yes       yes      yes
17   8960000   8500         3          2        4      yes        no       no
18   8890000   4600         3          2        2      yes       yes       no
19   8855000   6420         3          2        2      yes        no       no

   hotwaterheating airconditioning  parking prefarea furnishingstatus  Cluster
0               no              yes        2      yes        furnished        1
1               no              yes        3       no        furnished        1
2               no               no        2      yes   semi-furnished        2
3               no              yes        3      yes        furnished        1
4               no              yes        2       no        furnished        2
5               no              yes        2      yes   semi-furnished        1
6               no              yes        2      yes   semi-furnished        1
7               no               no        0       no      unfurnished        1
8               no              yes        2      yes        furnished        2
9               no              yes        1      yes      unfurnished        1
10              no              yes        2      yes        furnished        2
11             yes               no        2       no   semi-furnished        1
12              no              yes        1      yes   semi-furnished        1
13             yes               no        2       no        furnished        1
14              no               no        0      yes   semi-furnished        1
15              no               no        2       no   semi-furnished        2
16              no              yes        1      yes      unfurnished        1
17              no              yes        2       no        furnished        1
18              no              yes        2       no        furnished        1
19              no              yes        1      yes   semi-furnished        1
```

Ex5: Exploratory Data Analysis

Learning Objective:

The objective of this experiment is to enable students to understand and perform Exploratory Data Analysis (EDA)on a real-world Netflix titles dataset using Python.

Problem Statement:

Netflix maintains a large collection of movies and TV shows containing information such as title, content type, release year, date added, country, rating, duration, genre, director, cast, and description. Netflix wants to understand the characteristics and growth of its content library and identify important patterns in its movies and TV shows.

As a Data Scientist, perform an Exploratory Data Analysis on the Netflix Titles dataset. Analyse the content distribution, ratings, release and addition trends, duration, countries, genres, directors, actors, and textual descriptions. Use suitable statistical techniques and visualisations to identify patterns and generate meaningful insights from the data.

Tasks to Perform

: 1. Download Netflix_titles dataset from kaggle and convert it to dataframe.

2. Explore the Dataset

3. Content Distribution Analysis. (Plot graphs wherever needed)

a. How many Movies vs TV Shows are in the dataset?

b. What is the distribution of ratings (e.g., TV-MA, PG, R, etc.)?

c. Which countries have the most Netflix content?

4. Time-Based Trend Analysis (Plot graphs wherever needed) a. In which years were the most shows/movies released? b. In which years were titles added to Netflix? c. How has Netflix's content growth trend changed over time?

5. Duration & Content Type Analysis (Plot graphs wherever needed) a. What is the distribution of movie durations (in minutes)? b. How many TV shows exist per number of seasons?

6. Genre & Categories Analysis (Plot graphs wherever needed) a. What are the top 10 most common genres on Netflix? b. Which directors have the most titles on Netflix? c. Which actors appear most frequently across Netflix shows/movies?

7. Text & Descriptions a. What are the most common words in the description field? (word cloud)

8. In which months of the year does Netflix add the most content?

9. Which countries produce more movies vs TV shows?

10. What are the most common ratings for Movies vs TV Shows?

11. What are the oldest and newest titles available on Netflix?

12. Which genres frequently co-occur together? (Genre correlation heatmap)

```python
#Experiment 5
#Nivass C R[241801193]

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
from collections import Counter
from wordcloud import WordCloud

sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)
```

```python
#Nivass C R[241801193]
#step-1
from google.colab import files

uploaded = files.upload()
df = pd.read_csv("netflix_titles.csv")

print("Dataset Shape:", df.shape)
df.head()
```

```
Choose files  No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving netflix_titles.csv to netflix_titles.csv
Dataset Shape: (8807, 12)
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, TV Dramas, TV Mysteries	After crossing paths at a party, a Cape Town t...
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	2021	TV-MA	1 Season	Crime TV Shows, International TV Shows, TV Act...	To protect his family from a powerful drug lor...
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2021	TV-MA	1 Season	Docuseries, Reality TV	Feuds, flirtations and toilet talk go down amo...
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, Romantic TV Shows, TV ...	In a city of coaching centers known to train I...

```python
#Nivass C R[241801193]
#step-2
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])

print("\nColumn Names:")
print(df.columns.tolist())

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Rows:", df.duplicated().sum())
df.describe(include="all").T
```

```
Rows: 8807
Columns: 12

Column Names:
['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added', 'release_year', 'rating', 'duration', 'listed_in', 'description']

Data Types:
show_id         object
type            object
title           object
director        object
cast            object
country         object
date_added      object
release_year     int64
rating          object
duration        object
listed_in       object
description     object
dtype: object

Missing Values:
show_id            0
type               0
title              0
director        2634
cast             825
country          831
date_added        10
release_year       0
rating             4
duration           3
listed_in          0
```

```
Duplicate Rows: 0
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
show_id	8807	8807	s8807	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
type	8807	2	Movie	6131	NaN	NaN	NaN	NaN	NaN	NaN	NaN
title	8807	8807	Zubaan	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
director	6173	4528	Rajiv Chilaka	19	NaN	NaN	NaN	NaN	NaN	NaN	NaN
cast	7982	7692	David Attenborough	19	NaN	NaN	NaN	NaN	NaN	NaN	NaN
country	7976	748	United States	2818	NaN	NaN	NaN	NaN	NaN	NaN	NaN
date_added	8797	1767	January 1, 2020	109	NaN	NaN	NaN	NaN	NaN	NaN	NaN
release_year	8807.0	NaN	NaN	NaN	2014.180198	8.819312	1925.0	2013.0	2017.0	2019.0	2021.0
rating	8803	17	TV-MA	3207	NaN	NaN	NaN	NaN	NaN	NaN	NaN
duration	8804	220	1 Season	1793	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```python
#Nivass C R[241801193]
#step-3
content_count = df["type"].value_counts()

print(content_count)

plt.figure(figsize=(7,5))
sns.countplot(data=df, x="type")
plt.title("Movies vs TV Shows")
plt.xlabel("Content Type")
plt.ylabel("Number of Titles")
plt.show()
rating_count = df["rating"].value_counts()

print(rating_count)

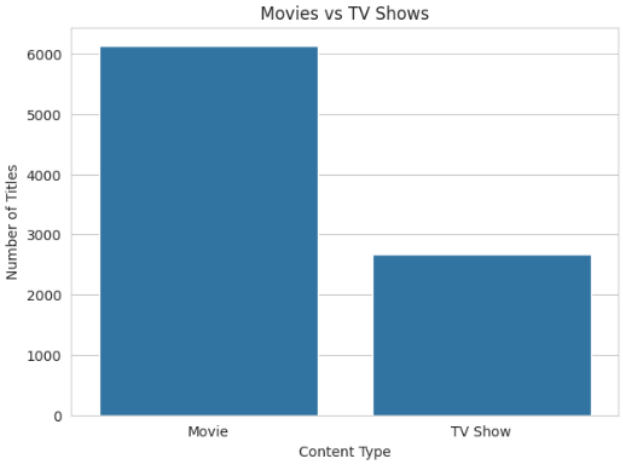
plt.figure(figsize=(10,6))
sns.barplot(x=rating_count.values, y=rating_count.index)
plt.title("Netflix Content Ratings")
plt.xlabel("Number of Titles")
plt.ylabel("Rating")
plt.show()
countries = (
    df["country"]
    .dropna()
    .str.split(", ")
    .explode()
    .value_counts()
)

print(countries.head(10))

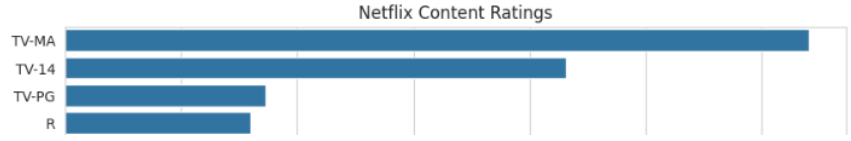
plt.figure(figsize=(10,6))
sns.barplot(x=countries.head(10).values,
            y=countries.head(10).index)

plt.title("Top 10 Countries Producing Netflix Content")
plt.xlabel("Number of Titles")
plt.ylabel("Country")
plt.show()
```

```
Movie      6131
TV Show    2676
Name: count, dtype: int64
```




```
rating
TV-MA       3207
TV-14       2160
TV-PG        863
R            799
PG-13        490
TV-Y7        334
TV-Y         307
PG           287
TV-G         220
NR            80
G             41
TV-Y7-FV       6
NC-17          3
UR             3
74 min         1
84 min         1
66 min         1
Name: count, dtype: int64
```



```python
#Nivass C R[241801193]
#step-4
release_years = df["release_year"].value_counts().sort_index()

plt.figure(figsize=(12,6))
plt.plot(release_years.index, release_years.values)
plt.title("Netflix Titles by Release Year")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.show()

print("Top Release Years:")
print(df["release_year"].value_counts().head(10))
df["date_added"] = pd.to_datetime(df["date_added"], errors="coerce")

added_year = df["date_added"].dt.year.value_counts().sort_index()

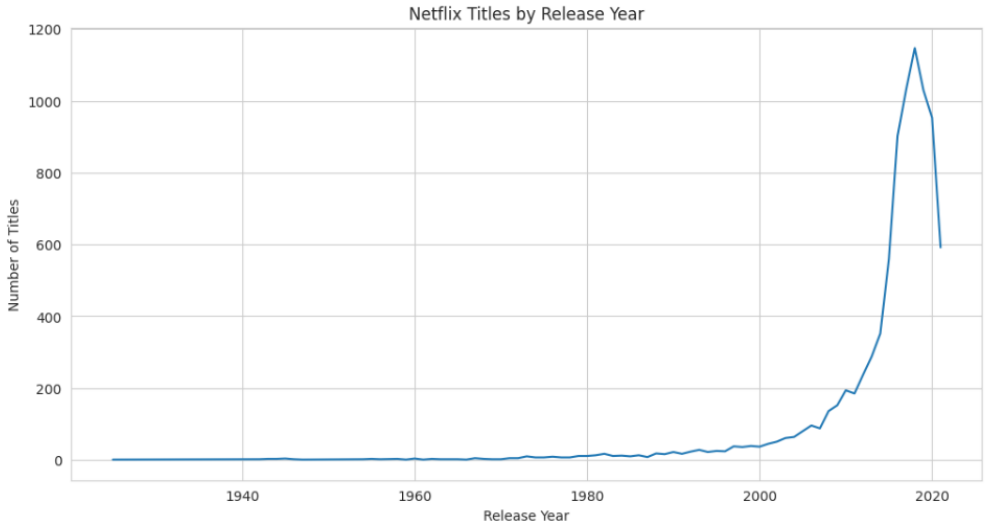
plt.figure(figsize=(10,6))
plt.plot(added_year.index, added_year.values, marker="o")
plt.title("Netflix Titles Added by Year")
plt.xlabel("Year")
plt.ylabel("Number of Titles Added")
plt.show()
growth = df.dropna(subset=["date_added"]).copy()

growth["year_added"] = growth["date_added"].dt.year

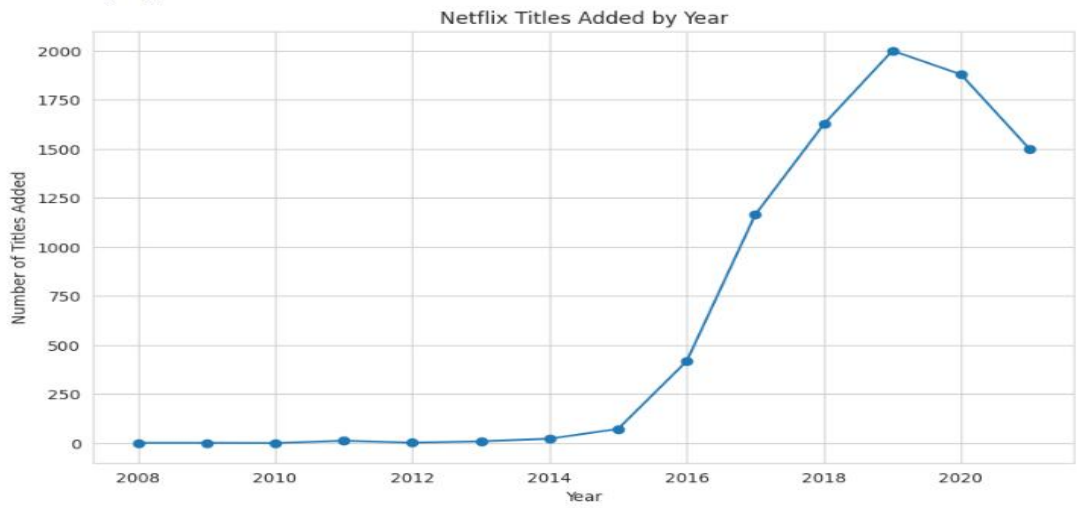
growth_count = growth.groupby("year_added").size().sort_index()
cumulative_growth = growth_count.cumsum()

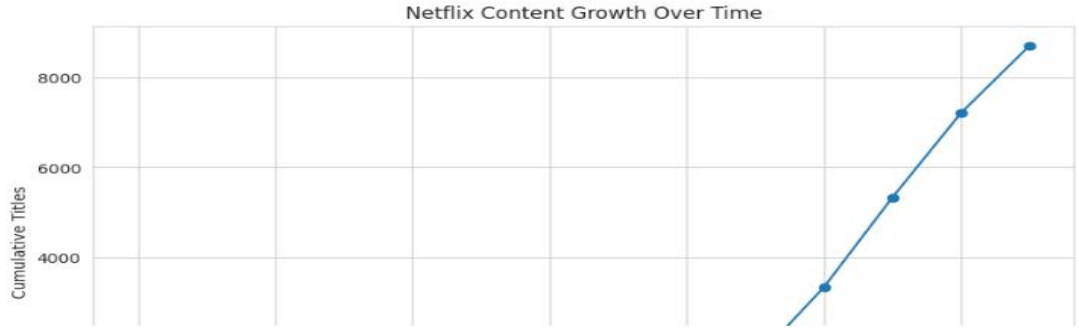
plt.figure(figsize=(10,6))
plt.plot(cumulative_growth.index,
         cumulative_growth.values,
         marker="o")

plt.title("Netflix Content Growth Over Time")
plt.xlabel("Year")
plt.ylabel("Cumulative Titles")
plt.show()
```




```
Top Release Years:
release_year
2018    1147
2017    1032
2019    1030
2020     953
2016     902
2021     592
2015     560
2014     352
2013     288
2012     237
```






```
#Nivass C R[241801193]
#step-5
movies = df[df["type"] == "Movie"].copy()

movies["duration_min"] = (
    movies["duration"]
    .str.extract(r"(\d+)")
    .astype(float)
)

print(movies["duration_min"].describe())

plt.figure(figsize=(10,6))
sns.histplot(movies["duration_min"].dropna(), bins=30, kde=True)

plt.title("Distribution of Movie Durations")
plt.xlabel("Duration (minutes)")
plt.ylabel("Number of Movies")
plt.show()
tv = df[df["type"] == "TV Show"].copy()

tv["seasons"] = (
    tv["duration"]
    .str.extract(r"(\d+)")
    .astype(float)
)

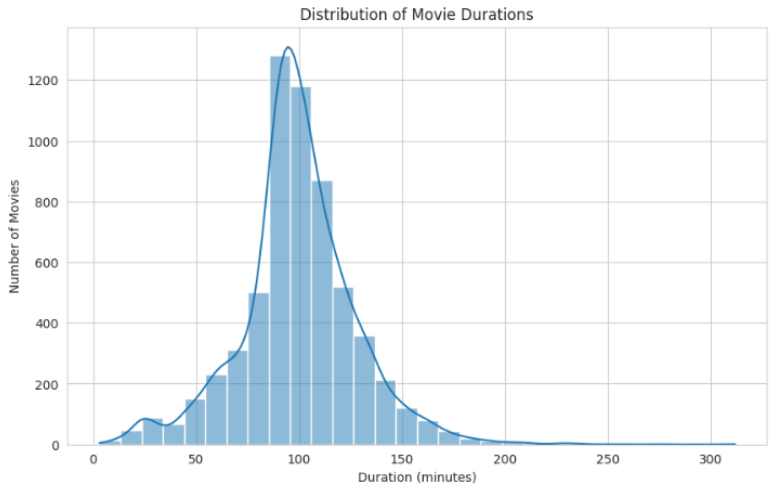
season_count = tv["seasons"].value_counts().sort_index()

print(season_count)

plt.figure(figsize=(10,6))
sns.barplot(x=season_count.index.astype(str),
            y=season_count.values)

plt.title("TV Shows by Number of Seasons")
plt.xlabel("Number of Seasons")
plt.ylabel("Number of TV Shows")
plt.show()
```

```
count    6128.000000
mean       99.577187
std        28.290593
min         3.000000
25%        87.000000
50%        98.000000
75%       114.000000
max       312.000000
Name: duration_min, dtype: float64
```




```
seasons
1.0     1793
2.0      425
3.0      199
4.0       95
5.0       65
6.0       33
7.0       23
8.0       17
9.0        9
10.0       7
11.0       2
12.0       2
13.0       3
15.0       2
17.0       1
Name: count, dtype: int64
```

```
#Nivass C R[241801193]
#step-6
genres = (
    df["listed_in"]
    .dropna()
    .str.split(", ")
    .explode()
    .value_counts()
)

print(genres.head(10))

plt.figure(figsize=(10,6))
sns.barplot(x=genres.head(10).values,
            y=genres.head(10).index)

plt.title("Top 10 Most Common Netflix Genres")
plt.xlabel("Number of Titles")
plt.ylabel("Genre")
plt.show()
directors = (
    df["director"]
    .dropna()
    .str.split(", ")
    .explode()
    .value_counts()
)

print(directors.head(10))

plt.figure(figsize=(10,6))
sns.barplot(x=directors.head(10).values,
            y=directors.head(10).index)

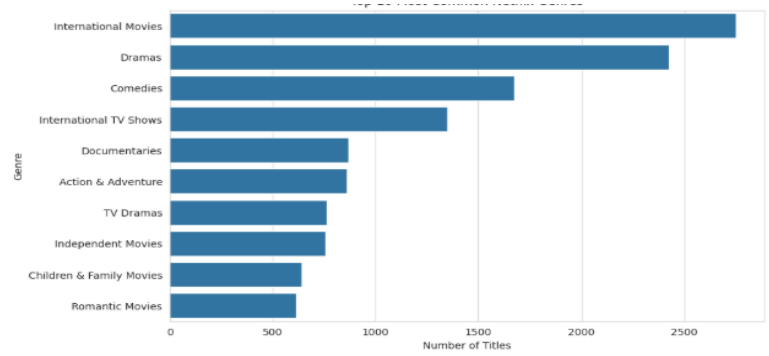
plt.title("Top 10 Directors on Netflix")
plt.xlabel("Number of Titles")
plt.ylabel("Director")
plt.show()
actors = (
    df["cast"]
    .dropna()
    .str.split(", ")
    .explode()
    .value_counts()
)

print(actors.head(10))

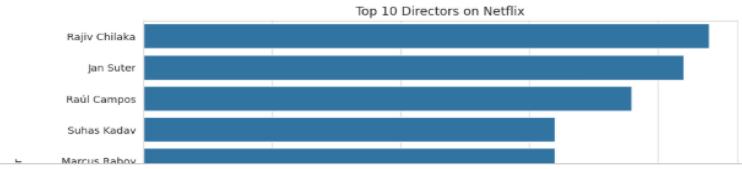
plt.figure(figsize=(10,6))
sns.barplot(x=actors.head(10).values,
            y=actors.head(10).index)

plt.title("Top 10 Most Frequent Actors")
plt.xlabel("Number of Titles")
plt.ylabel("Actor")
plt.show()
```

```
listed_in
International Movies        2752
Dramas                      2427
Comedies                    1674
International TV Shows      1351
Documentaries                869
Action & Adventure           859
TV Dramas                    763
Independent Movies           756
Children & Family Movies     641
Romantic Movies              616
Name: count, dtype: int64
```



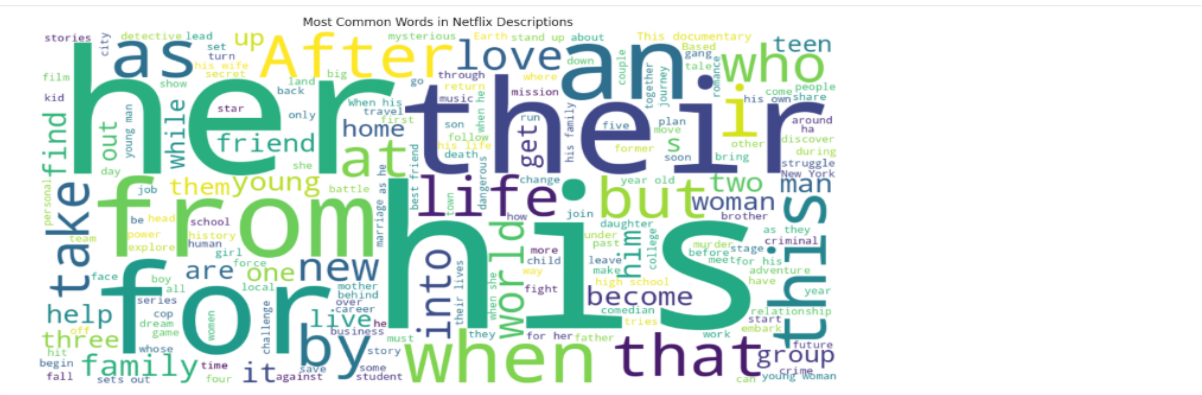

```
director
Rajiv Chilaka             22
Jan Suter                 21
Raúl Campos               19
Suhas Kadav               16
Marcus Raboy              16
Jay Karas                 15
Cathy Garcia-Molina       13
Martin Scorsese           12
Youssef Chahine           12
Jay Chapman               12
Name: count, dtype: int64
```




```
#Nivass C R[241801193]
#step-7
text = " ".join(df["description"].dropna().astype(str))

wordcloud = WordCloud(
    width=1200,
    height=600,
    background_color="white",
    stopwords=set(["the", "and", "of", "to", "a", "in", "with", "on"])
).generate(text)

plt.figure(figsize=(15,7))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.title("Most Common Words in Netflix Descriptions")
plt.show()
```




```
#Nivass C R[241801193]
#step-8
month_count = (
    df["date_added"]
    .dt.month_name()
    .value_counts()
)

month_order = [
    "January", "February", "March", "April",
    "May", "June", "July", "August",
    "September", "October", "November", "December"
]

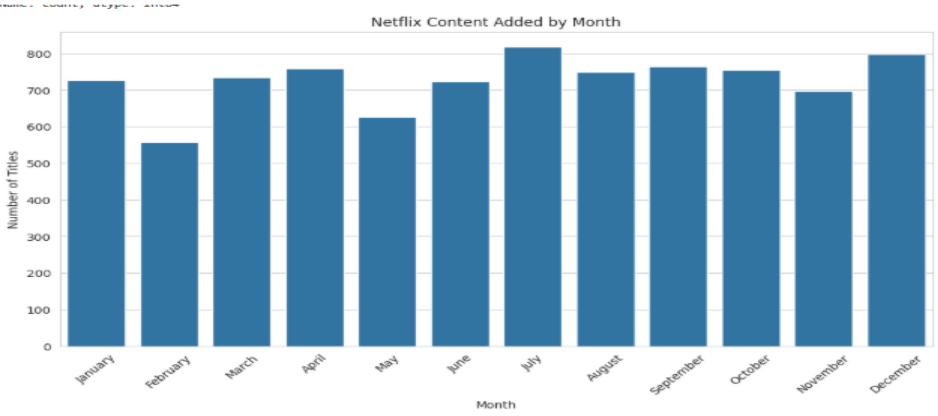
month_count = month_count.reindex(month_order)

print(month_count)

plt.figure(figsize=(12,6))
sns.barplot(x=month_count.index,
            y=month_count.values)

plt.title("Netflix Content Added by Month")
plt.xlabel("Month")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```

```
date_added
January      727
February     557
March        734
April        759
May          626
June         724
July         819
August       749
September    765
October      755
November     697
December     797
Name: count, dtype: int64
```




```
#Nivass C R[241801193]
#step-9
country_type = df.dropna(subset=["country"]).copy()

country_type["country"] = country_type["country"].str.split(", ")
country_type = country_type.explode("country")

top_countries = (
    country_type["country"]
    .value_counts()
    .head(10)
    .index
)

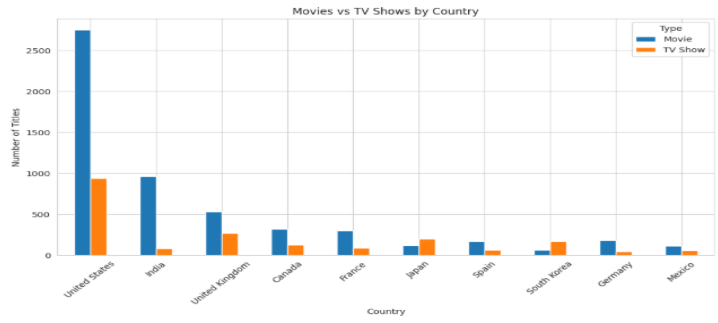
country_comparison = pd.crosstab(
    country_type["country"],
    country_type["type"]
).loc[top_countries]

print(country_comparison)

country_comparison.plot(
    kind="bar",
    figsize=(12,6)
)

plt.title("Movies vs TV Shows by Country")
plt.xlabel("Country")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.legend(title="Type")
plt.show()
```

```
type             Movie  TV Show
country
United States     2751      938
India              962       84
United Kingdom     532      272
Canada             319      126
France             303       90
Japan              119      199
Spain              171       61
South Korea         61      170
Germany            182       44
Mexico             111       58
```

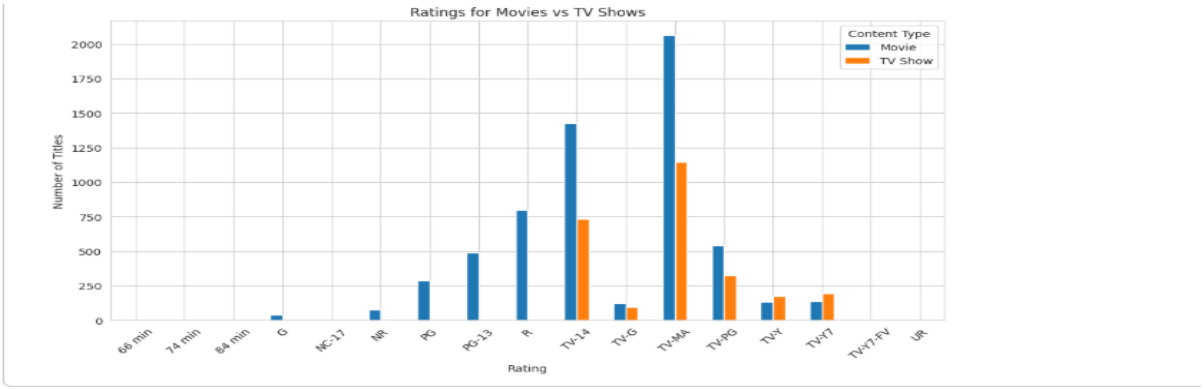

```
#Nivass C R[241801193]
#step-10
rating_type = pd.crosstab(
    df["rating"],
    df["type"]
)

print(rating_type)

rating_type.plot(
    kind="bar",
    figsize=(12,6)
)

plt.title("Ratings for Movies vs TV Shows")
plt.xlabel("Rating")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.legend(title="Content Type")
plt.show()
```

```
type      Movie  TV Show
rating
66 min        1        0
74 min        1        0
84 min        1        0
G            41        0
NC-17         3        0
NR           75        5
PG          287        0
PG-13       490        0
R           797        2
TV-14      1427      733
TV-G        126       94
TV-MA      2062     1145
TV-PG       540      323
TV-Y        131      176
TV-Y7       139      195
TV-Y7-FV      5        1
UR            3        0
```




```
#Nivass C R[241801193]
#step-11
oldest = df.loc[df["release_year"].idxmin()]
newest = df.loc[df["release_year"].idxmax()]

print("OLDEST TITLE")
print("Title:", oldest["title"])
print("Release Year:", oldest["release_year"])
print("Type:", oldest["type"])

print("\nNEWEST TITLE")
print("Title:", newest["title"])
print("Release Year:", newest["release_year"])
print("Type:", newest["type"])
```

```
OLDEST TITLE
Title: Pioneers: First Women Filmmakers*
Release Year: 1925
Type: TV Show

NEWEST TITLE
Title: Blood & Water
Release Year: 2021
Type: TV Show
```

```
#Nivass C R[241801193]
#step-12
genre_listx = (
    df["listed_in"]
    .dropna()
    .str.split(", ")
)

top_genres = genres.head(10).index.tolist()

genre_matrix = pd.DataFrame(
    0,
    index=top_genres,
    columns=top_genres
)

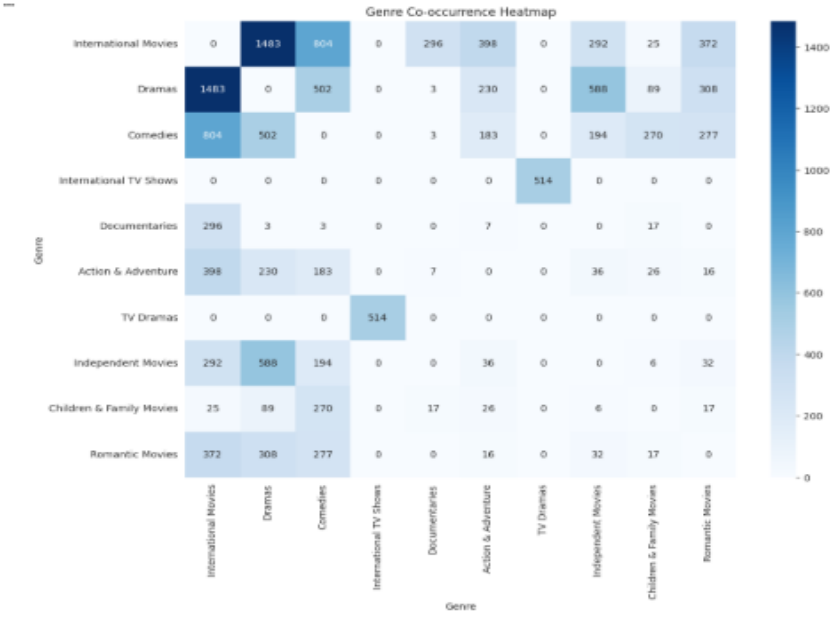
for genre_list in genre_listx:
    selected = [g for g in genre_list if g in top_genres]

    for g1 in selected:
        for g2 in selected:
            if g1 != g2:
                genre_matrix.loc[g1, g2] += 1

genre_matrix.head()
plt.figure(figsize=(12,9))

sns.heatmap(
    genre_matrix,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.title("Genre Co-occurrence Heatmap")
plt.xlabel("Genre")
plt.ylabel("Genre")
plt.show()
```



```python
#Nivass C R[241801193]
#step-13
print("========== NETFLIX EDA SUMMARY ==========\n")

print("Total Titles:", len(df))

print("\nContent Distribution:")
print(df["type"].value_counts())

print("\nTop 5 Countries:")
print(countries.head(5))

print("\nTop 5 Genres:")
print(genres.head(5))

print("\nTop 5 Directors:")
print(directors.head(5))

print("\nTop 5 Actors:")
print(actors.head(5))

print("\nMost Common Ratings:")
print(df["rating"].value_counts().head(5))

print("\nOldest Release Year:", df["release_year"].min())
print("Newest Release Year:", df["release_year"].max())

print("\nMonth with Most Additions:",
      month_count.idxmax())

print("========================================")
```

```
========== NETFLIX EDA SUMMARY ==========

Total Titles: 8807

Content Distribution:
type
Movie      6131
TV Show    2676
Name: count, dtype: int64

Top 5 Countries:
country
United States     3689
India             1046
United Kingdom     804
Canada             445
France             393
Name: count, dtype: int64

Top 5 Genres:
listed_in
International Movies      2752
Dramas                    2427
Comedies                  1674
International TV Shows    1351
Documentaries              869
Name: count, dtype: int64

Top 5 Directors:
director
Rajiv Chilaka    22
Jan Suter        21
Raúl Campos      19
Suhas Kadav      16
Marcus Raboy     16
Name: count, dtype: int64

Top 5 Actors:
cast
Anupam Kher          43
Shah Rukh Khan       35
Julie Tejwani        33
Takahiro Sakurai     32
Naseeruddin Shah     32
Name: count, dtype: int64

Most Common Ratings:
rating
```

Ex6: Time Series Data Analysis and Forecast

Learning Objective: The objective of this experiment is to enable students to understand and implement the fundamental concepts of time series data analysis and forecasting using Python.

Problem Statement:

An airline company maintains monthly records of the number of passengers travelling by air. The company wants to understand historical passenger trends and seasonal patterns and use this information to forecast future passenger demand.

As a Data Scientist, analyse the historical monthly airline passenger data, determine whether the time series is stationary, apply appropriate transformations when necessary, build an ARIMA forecasting model, predict passenger numbers for the test period, and evaluate the accuracy of the forecasts.

Tasks to Perform:

1. Download airline-passenges dataset from https://github.com/jbrownlee/Datasets/blob/master/airline-passengers.csv

2. Read the CSV with parse_dates and index_col so the time index becomes a DatetimeIndex.

3. Explore the Dataset.

4. Plot the series to inspect trend/seasonality/outliers.

5. Do Stationarity check by running Augmented Dickey-Fuller (ADF) test.

6. If the series is non-stationary, convert it to stationary by differencing the series.

7. Re-run ADF on the differenced series and visualize the new dataframe.

8. If still non-stationary, consider another difference (d=2) or transformation (log) before differencing.

9. Split the dataset for training and testing on original series.

10. Fit ARIMA model on training set. d value comes from Step 8 (how many differences you applied). 11. Perform forecasts on the test dataset.

12. Compute metrics MSE, MAE, RMSE on the predictions.

13. Plot train, test, and predictions in a single plot.

```python
#Nivass C R[241801193]
#Experiment-6
#Step -1
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"
df = pd.read_csv(url, parse_dates=["Month"], index_col="Month")
print("Dataset downloaded successfully")
print(df.head())
```

```
Dataset downloaded successfully
            Passengers
Month
1949-01-01         112
1949-02-01         118
1949-03-01         132
1949-04-01         129
1949-05-01         121
```

```python
#Nivass C R[241801193]
#Experiment-6
#Step -2
df.index = pd.DatetimeIndex(df.index, freq="MS")
df.columns = ["Passengers"]
print("Dataset information:")
print(df.info())
print("\nDate range:", df.index.min(), "to", df.index.max())
print("Time index type:", type(df.index))
```

```
Dataset information:
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 144 entries, 1949-01-01 to 1960-12-01
Freq: MS
Data columns (total 1 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Passengers  144 non-null    int64
dtypes: int64(1)
memory usage: 2.2 KB
None

Date range: 1949-01-01 00:00:00 to 1960-12-01 00:00:00
Time index type: <class 'pandas.core.indexes.datetimes.DatetimeIndex'>
```

```python
#Nivass C R[241801193]
#Experiment-6
#Step -3

print("First five records:")
print(df.head())

print("\nLast five records:")
print(df.tail())

print("\nDataset shape:", df.shape)

print("\nStatistical summary:")
print(df.describe())

print("\nMissing values:")
print(df.isnull().sum())
```

```
First five records:
            Passengers
Month
1949-01-01         112
1949-02-01         118
1949-03-01         132
1949-04-01         129
1949-05-01         121

Last five records:
            Passengers
Month
1960-08-01         606
1960-09-01         508
1960-10-01         461
1960-11-01         390
1960-12-01         432

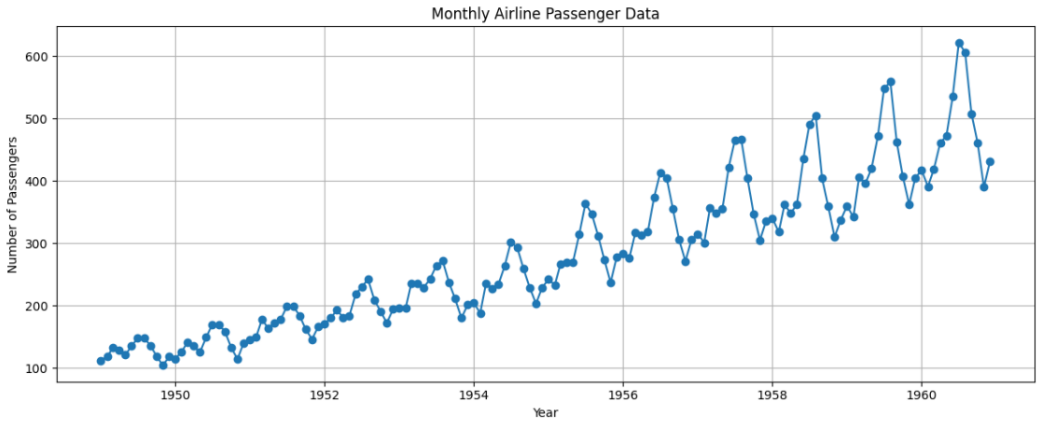
Dataset shape: (144, 1)

Statistical summary:
       Passengers
count  144.000000
mean   280.298611
std    119.966317
min    104.000000
25%    180.000000
50%    265.500000
75%    360.500000
max    622.000000

Missing values:
Passengers    0
dtype: int64
```

```python
#Nivass C R[241801193]
#Experiment-6
#Step -4

plt.figure(figsize=(12, 5))
plt.plot(df.index, df["Passengers"], marker="o")
plt.title("Monthly Airline Passenger Data")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.grid(True)
plt.tight_layout()
plt.show()
```




```python
#Nivass C R[241801193]
#Experiment-6
#Step -5

from statsmodels.tsa.stattools import adfuller

result = adfuller(df["Passengers"].dropna())

print("ADF Statistic:", result[0])
print("p-value:", result[1])
print("Critical Values:", result[4])

if result[1] <= 0.05:
    print("The series is stationary.")
else:
    print("The series is non-stationary.")
```

```
ADF Statistic: 0.8153688792060498
p-value: 0.991880243437641
Critical Values: {'1%': np.float64(-3.4816817173418295), '5%': np.float64(-2.8840418343195267), '10%': np.float64(-2.578770059171598)}
The series is non-stationary.
/tmp/ipykernel_1102/1308921461.py:7: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after July 2027, whichever is later, the default behavior will switch to
  result = adfuller(df["Passengers"].dropna())
```

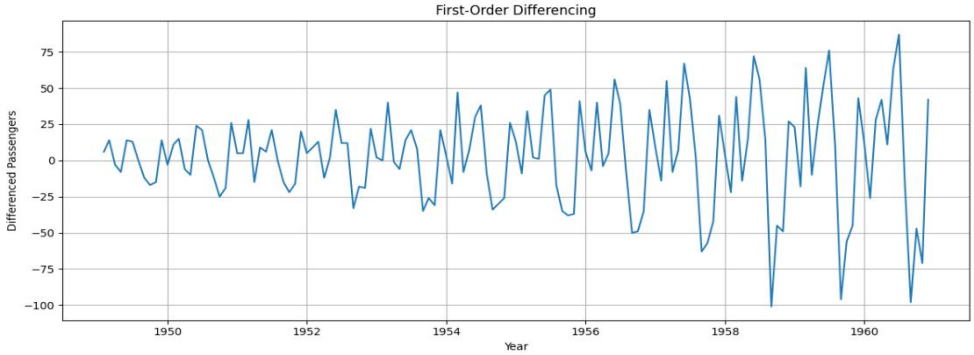
```python
#Nivass C R[241801193]
#Experiment-6
#Step -6

df["First_Difference"] = df["Passengers"].diff()

print(df[["Passengers", "First_Difference"]].head(10))

plt.figure(figsize=(12, 5))
plt.plot(df.index, df["First_Difference"])
plt.title("First-Order Differencing")
plt.xlabel("Year")
plt.ylabel("Differenced Passengers")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
            Passengers  First_Difference
Month
1949-01-01         112               NaN
1949-02-01         118               6.0
1949-03-01         132              14.0
1949-04-01         129              -3.0
1949-05-01         121              -8.0
1949-06-01         135              14.0
1949-07-01         148              13.0
1949-08-01         148               0.0
1949-09-01         136             -12.0
1949-10-01         119             -17.0
```



```python
#Nivass C R[241801193]
#Experiment-6
#Step -7

diff_series = df["First_Difference"].dropna()
result_diff = adfuller(diff_series)

print("ADF Statistic:", result_diff[0])
print("p-value:", result_diff[1])
print("Critical Values:", result_diff[4])

if result_diff[1] <= 0.05:
    print("The differenced series is stationary.")
else:
    print("The differenced series is still non-stationary.")
```

```
ADF Statistic: -2.8292668241700047
p-value: 0.05421329028382478
Critical Values: {'1%': np.float64(-3.4816817173418295), '5%': np.float64(-2.8840418343195267), '10%': np.float64(-2.578770059171598)}
The differenced series is still non-stationary.
/tmp/ipykernel_1102/3183178890.py:6: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after July 2027, whichever is later, the default behavior will switch to always returning an ADFullerRes
  result_diff = adfuller(diff_series)
```

```python
#Nivass C R[241801193]
#Experiment-6
#Step -8

log_series = np.log(df["Passengers"])

first_diff = log_series.diff().dropna()
adf_first = adfuller(first_diff)

print("First difference of log series:")
print("ADF Statistic:", adf_first[0])
print("p-value:", adf_first[1])

if adf_first[1] <= 0.05:
    d = 1
    stationary_series = first_diff
else:
    second_diff = first_diff.diff().dropna()
    adf_second = adfuller(second_diff)

    print("\nSecond difference of log series:")
    print("ADF Statistic:", adf_second[0])
    print("p-value:", adf_second[1])

    d = 2
    stationary_series = second_diff

print("\nSelected differencing order d =", d)

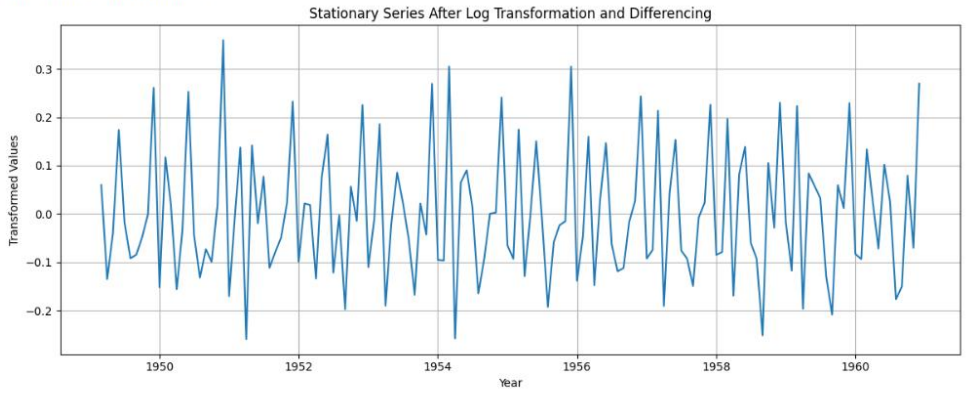
plt.figure(figsize=(12, 5))
plt.plot(stationary_series.index, stationary_series)
plt.title("Stationary Series After Log Transformation and Differencing")
plt.xlabel("Year")
plt.ylabel("Transformed Values")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_1102/927703430.py:8: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after July 2027, whichever is later, the default behavior will switch to always returning an ADFullerResul
  adf_first = adfuller(first_diff)
/tmp/ipykernel_1102/927703430.py:19: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after July 2027, whichever is later, the default behavior will switch to always returning an ADFullerResu
  adf_second = adfuller(second_diff)
First difference of log series:
ADF Statistic: -2.717130983881114
p-value: 0.07112054815086184

Second difference of log series:
ADF Statistic: -8.196629132182246
p-value: 7.419304549376186e-13

Selected differencing order d = 2
```

Selected differencing order: d = 2




```python
#Nivass C R[241801193]
#Experiment-6
#Step -9

train_size = len(df) - 12

train = df["Passengers"].iloc[:train_size]
test = df["Passengers"].iloc[train_size:]

print("Training data size:", len(train))
print("Testing data size:", len(test))

print("\nTraining period:", train.index.min(), "to", train.index.max())
print("Testing period:", test.index.min(), "to", test.index.max())
```

```
Training data size: 132
Testing data size: 12

Training period: 1949-01-01 00:00:00 to 1959-12-01 00:00:00
Testing period: 1960-01-01 00:00:00 to 1960-12-01 00:00:00
```

```python
#Nivass C R[241801193]
#Experiment-6
#Step -10

from statsmodels.tsa.arima.model import ARIMA

model = ARIMA(train, order=(2, d, 2))
model_fit = model.fit()

print(model_fit.summary())
```

```
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/statespace/mlemodel.py:679: EstimationWarning: Non-invertible starting MA parameters found. Using zeros as starting parameters.
  start_params = self.start_params
                               SARIMAX Results
==============================================================================
Dep. Variable:             Passengers   No. Observations:                  132
Model:                 ARIMA(2, 2, 2)   Log Likelihood                -622.495
Date:                Fri, 09 Oct 2026   AIC                           1254.990
Time:                        05:22:48   BIC                           1269.327
Sample:                    01-01-1949   HQIC                          1260.815
                         - 12-01-1959
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.5057      0.113     -4.477      0.000      -0.727      -0.284
ar.L2          0.1090      0.109      1.002      0.316      -0.104       0.322
ma.L1         -0.0556      9.029     -0.006      0.995     -17.751      17.640
ma.L2         -0.9442      8.553     -0.110      0.912     -17.708      15.820
sigma2       812.0365   7370.071      0.110      0.912   -1.36e+04    1.53e+04
===================================================================================
Ljung-Box (L1) (Q):                   0.00   Jarque-Bera (JB):                 2.95
Prob(Q):                              0.96   Prob(JB):                         0.23
Heteroskedasticity (H):               6.75   Skew:                            -0.24
Prob(H) (two-sided):                  0.00   Kurtosis:                         3.57
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

```python
#Nivass C R[241801193]
#Experiment-6
#Step -11

forecast_result = model_fit.get_forecast(steps=len(test))

predictions = forecast_result.predicted_mean
predictions.index = test.index

print("Actual passenger numbers:")
print(test)

print("\nForecasted passenger numbers:")
print(predictions)
```

```
Actual passenger numbers:
Month
1960-01-01    417
1960-02-01    391
1960-03-01    419
1960-04-01    461
1960-05-01    472
1960-06-01    535
1960-07-01    622
1960-08-01    606
1960-09-01    508
1960-10-01    461
1960-11-01    390
1960-12-01    432
Freq: MS, Name: Passengers, dtype: int64

Forecasted passenger numbers:
Month
1960-01-01    436.476189
1960-02-01    428.670343
1960-03-01    439.475862
1960-04-01    436.585972
1960-05-01    442.650938
1960-06-01    442.694861
1960-07-01    446.759855
1960-08-01    448.134941
1960-09-01    451.308716
1960-10-01    453.279623
1960-11-01    456.054925
1960-12-01    458.292301
Freq: MS, Name: predicted_mean, dtype: float64
```

```python
#Nivass C R[241801193]
#Experiment-6
#Step -12

from sklearn.metrics import mean_squared_error, mean_absolute_error

mse = mean_squared_error(test, predictions)
mae = mean_absolute_error(test, predictions)
rmse = np.sqrt(mse)

print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)
```

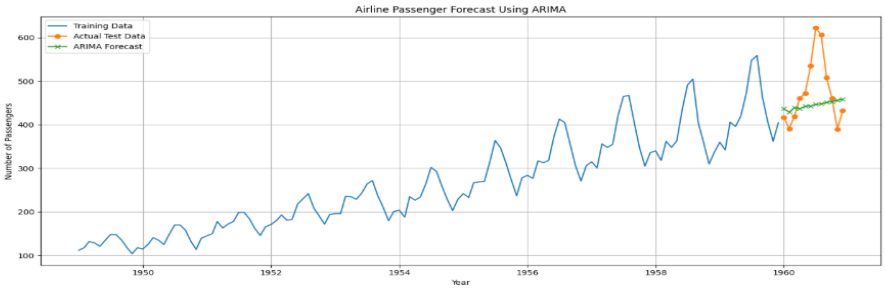
```
Mean Squared Error (MSE): 6346.148768370937
Mean Absolute Error (MAE): 59.462826130193186
Root Mean Squared Error (RMSE): 79.66271881106658
```

```python
#Nivass C R[241801193]
#Experiment-6
#Step -13

plt.figure(figsize=(14, 6))

plt.plot(train.index, train, label="Training Data")
plt.plot(test.index, test, label="Actual Test Data", marker="o")
plt.plot(predictions.index, predictions, label="ARIMA Forecast", marker="x")

plt.title("Airline Passenger Forecast Using ARIMA")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



Ex7: Model Planning and Building - Clustering

Learning Objective:

The objective of this experiment is to enable students to understand and implement the K Means clustering algorithm for customer segmentation.

Problem Statement:

A retail company wants to group its customers based on their Annual Income and Spending Score to design personalized marketing campaigns. By identifying customers with similar purchasing behavior, the company can improve customer satisfaction, optimize promotional offers, and increase overall sales.

Your task is to analyze the customer dataset, apply the K-Means Clustering algorithm to identify customer segments, visualize the clusters, and recommend suitable marketing strategies for each segment.

Tasks to Perform:

1. Use Mall Customer Segmentation Dataset from Kaggle and load it into a Pandas DataFrame.

2. Explore the dataset by: Displaying the first and last five records, Checking the dataset shape and data types, Identifying missing values and duplicate records, Displaying summary statistics.

3. Select the important features for clustering: Annual Income (k$), Spending Score (1–100).

4. Standardize the selected features using StandardScaler for clustering.

5. Visualize the selected features using: Scatter Plot, Pair Plot.

Model Planning and Building

6. Determine the optimal number of clusters using the Elbow Method. Plot the Elbow Curve and identify the optimal value of K.

7. Train the K-Means Clustering model using the optimal number of clusters. Assign cluster labels to each customer, add as a new column in the dataset and save as a file.

Cluster Visualization

8. Visualize the customer clusters using a scatter plot: X-axis: Annual Income, Y-axis: Spending Score, Different colors for each cluster, Highlight cluster centroids, Display the coordinates of the cluster centroids.

Cluster Analysis

9. Calculate the number of customers in each cluster.

10. Compute the average Annual Income and Spending Score for each cluster.

```python
#Nivass C R[241801193]
#Ex7
#Step -1
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
```

```python
#Nivass C R[241801193]
#Ex7
#Step -2
from google.colab import files
uploaded = files.upload()
filename = next(iter(uploaded))
df = pd.read_csv(filename)
print("First Five Records:")
display(df.head())
print("Last Five Records:")
display(df.tail())
print("Dataset Shape:", df.shape)
print("\nData Types:")
print(df.dtypes)
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDuplicate Records:", df.duplicated().sum())
print("\nSummary Statistics:")
display(df.describe(include="all"))
```

```
Saving Mall_Customers.csv to Mall_Customers.csv
First Five Records:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
Last Five Records:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

```
Dataset Shape: (200, 5)

Data Types:
CustomerID                int64
Gender                   object
Age                       int64
Annual Income (k$)        int64
Spending Score (1-100)    int64
dtype: object


Missing Values:
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64

Duplicate Records: 0

Summary Statistics:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200	200.000000	200.000000	200.000000
unique	NaN	2	NaN	NaN	NaN
top	NaN	Female	NaN	NaN	NaN
freq	NaN	112	NaN	NaN	NaN
mean	100.500000	NaN	38.850000	60.560000	50.200000
std	57.879185	NaN	13.969007	26.264721	25.823522
min	1.000000	NaN	18.000000	15.000000	1.000000
25%	50.750000	NaN	28.750000	41.500000	34.750000
50%	100.500000	NaN	36.000000	61.500000	50.000000
75%	150.250000	NaN	49.000000	78.000000	73.000000
max	200.000000	NaN	70.000000	137.000000	99.000000

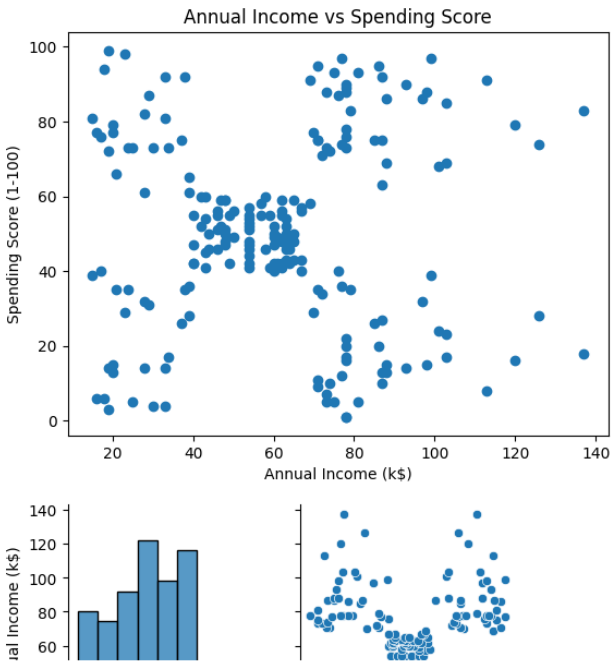
```
#Nivass C R[241801193]
#Ex7
#Step -3
X = df[["Annual Income (k$)", "Spending Score (1-100)"]]
display(X.head())
```

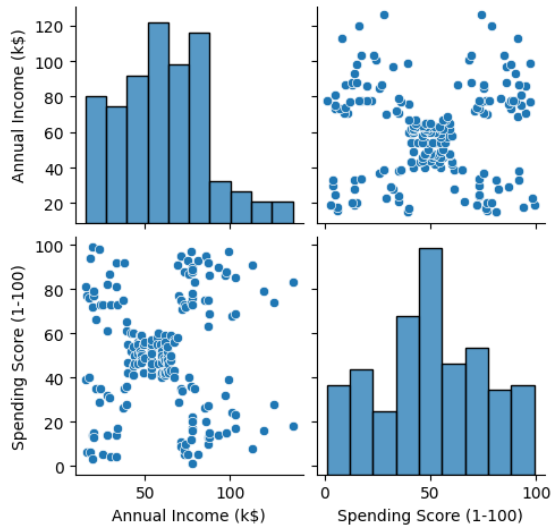
	Annual Income (k$)	Spending Score (1-100)
0	15	39
1	15	81
2	16	6
3	16	77
4	17	40

```
#Nivass C R[241801193]
#Ex7
#Step -4
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Standardized Features:")
print(X_scaled[:5])
```

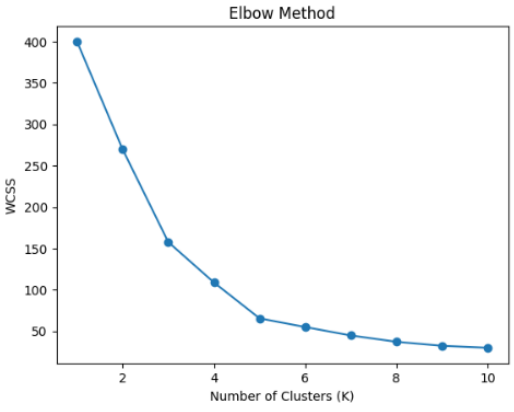
```
Standardized Features:
[[-1.73899919 -0.43480148]
 [-1.73899919  1.19570407]
 [-1.70082976 -1.71591298]
 [-1.70082976  1.04041783]
 [-1.66266033 -0.39597992]]
```

```
#Nivass C R[241801193]
#Ex7
#Step -5
plt.scatter(
    X["Annual Income (k$)"],
    X["Spending Score (1-100)"]
)
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Annual Income vs Spending Score")
plt.show()
sns.pairplot(X)
plt.show()
```






```
#Nivass C R[241801193]
#Ex7
#Step -6
wcss = []
for k in range(1, 11):
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)
plt.plot(range(1, 11), wcss, marker="o")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()
```




```
#Nivass C R[241801193]
#Ex7
#Step -7
kmeans = KMeans(
    n_clusters=5,
    random_state=42,
    n_init=10
)
df["Cluster"] = kmeans.fit_predict(X_scaled)
print("Cluster Labels:")
display(df.head())
df.to_csv("Mall_Customers_Clustered.csv", index=False)
files.download("Mall_Customers_Clustered.csv")
```

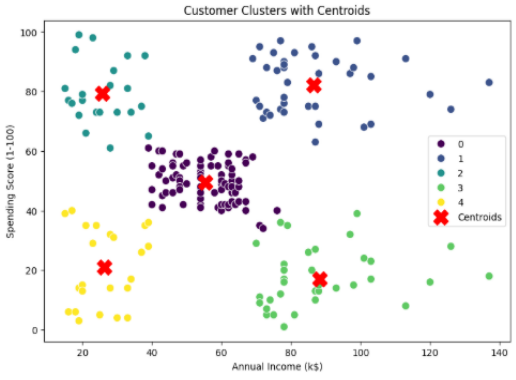
```
Cluster Labels:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	2
2	3	Female	20	16	6	4
3	4	Female	23	16	77	2
4	5	Female	31	17	40	4

```python
#Nivass C R[241801193]
#Ex7
#Step -8
centers = scaler.inverse_transform(kmeans.cluster_centers_)
plt.figure(figsize=(9, 6))
sns.scatterplot(
    data=df,
    x="Annual Income (k$)",
    y="Spending Score (1-100)",
    hue="Cluster",
    palette="viridis",
    s=70
)
plt.scatter(
    centers[:, 0],
    centers[:, 1],
    marker="X",
    s=250,
    color="red",
    label="Centroids"
)
for i, center in enumerate(centers):
    print(f"Cluster {i} Centroid: {center}")

plt.title("Customer Clusters with Centroids")
plt.legend()
plt.show()
```

```
Cluster 0 Centroid: [55.2962963  49.51851852]
Cluster 1 Centroid: [86.53846154 82.12820513]
Cluster 2 Centroid: [25.72727273 79.36363636]
Cluster 3 Centroid: [88.2        17.11428571]
Cluster 4 Centroid: [26.30434783 20.91304348]
```




```python
#Nivass C R[241801193]
#Ex7
#Step -9
print("Number of Customers in Each Cluster:")
print(df["Cluster"].value_counts().sort_index())
```

```
Number of Customers in Each Cluster:
Cluster
0    81
1    39
2    22
3    35
4    23
Name: count, dtype: int64
```

```python
#Nivass C R[241801193]
#Ex7
#Step -10
cluster_summary = df.groupby("Cluster")[
    ["Annual Income (k$)", "Spending Score (1-100)"]
].mean()
print("Average Income and Spending Score by Cluster:")
display(cluster_summary)
```

Average Income and Spending Score by Cluster:

Cluster	Annual Income (k$)	Spending Score (1-100)
0	55.296296	49.518519
1	86.538462	82.128205
2	25.727273	79.363636
3	88.200000	17.114286
4	26.304348	20.913043

Ex5: Information Extraction using spaCy

Learning Objective:

The objective of this experiment is to enable students to understand and implement information extraction techniques on unstructured customer review text using spaCy. Problem Statement: An e-commerce company receives thousands of customer reviews containing valuable information about products, brands, organizations, locations, quantities, and other entities. Manually identifying and organizing this information from large volumes of reviews is time consuming and inefficient

. As an NLP Engineer, your task is to process customer reviews using the spaCy Natural Language Processing library and develop an information extraction system that identifies and extracts meaningful linguistic information and named entities from the reviews. The extracted information should be organized into a structured format that can be further used for customer feedback analysis.

Tasks to Perform:

1. **Reuse the Amazon Fine Food Reviews Dataset used in the previous experiment and load it into a Pandas DataFrame. Install and import the required libraries.**

Clean the dataset

2. **Select the Text column containing customer reviews, Remove missing or null values, Remove duplicate reviews (if any), Retain only the first 10,000 reviews for faster processing.**

Text Preprocessing and Part-of-speech (POS) tags

3. Apply the spaCy pipeline to the review text. For any five reviews, display: Original review, Tokens, Lemmas, Part-of-speech (POS) tags.

4. Identify and display different POS categories such as: Nouns, Verbs, Adjectives, Adverbs.

5. Count and display the frequency of the major POS categories present in the reviews.

Named Entity Recognition (NER)

6. Apply Named Entity Recognition (NER) using spaCy to identify entities in the reviews. Extract and display entities such as: PERSON, ORG, DATE, MONEY, QUANTITY, PRODUCT, where recognized by the selected spaCy model. Display the extracted information in a structured DataFrame.

Information Extraction

7. Create a function that accepts a customer review as input and returns: Tokens, Lemmas, POS tags, Named entities, Entity labels.
8. Test the function using at least three new customer reviews.
9. Display the extracted information in a structured format.

```
#Nivass C R[241801193]
import pandas as pd
import spacy
from collections import Counter
```

```
#Nivass C R[241801193]
%pip install pandas spacy
!python -m spacy download en_core_web_sm
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: spacy in /usr/local/lib/python3.13/dist-packages (3.8.16)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.13/dist-packages (from pandas) (2.1.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas) (2026.4)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.13/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.13/dist-packages (from spacy) (3.0.13)
Requirement already satisfied: thinc<8.4.0,>=8.3.12 in /usr/local/lib/python3.13/dist-packages (from spacy) (8.3.13)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.5.3 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.5.4)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<2.0.0,>=1.0.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.0.0)
Requirement already satisfied: confection<2.0.0,>=1.3.2 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.3.3)
Requirement already satisfied: typer<1.0.0,>=0.3.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (0.27.2)
Requirement already satisfied: click<9.0.0,>=8.2.1 in /usr/local/lib/python3.13/dist-packages (from spacy) (8.5.0)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (4.67.3)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic<3.0.0,>=2.0.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.13.5)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.13/dist-packages (from spacy) (80.10.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (26.3)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.13/dist-packages (from pydantic<3.0.0,>=2.0.0->spacy) (0.8.0)
Requirement already satisfied: pydantic-core==2.46.5 in /usr/local/lib/python3.13/dist-packages (from pydantic<3.0.0,>=2.0.0->spacy) (2.46.5)
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.13/dist-packages (from pydantic<3.0.0,>=2.0.0->spacy) (4.16.0)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.13/dist-packages (from pydantic<3.0.0,>=2.0.0->spacy) (0.4.4)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.20)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2026.7.22)
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.13/dist-packages (from thinc<8.4.0,>=8.3.12->spacy) (1.3.3)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.13/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (1.5.4)
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.13/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.13/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (0.0.5)
Requirement already satisfied: cloudpathlib>=0.7.0 in /usr/local/lib/python3.13/dist-packages (from weasel<2.0.0,>=1.0.0->spacy) (0.25.0)
Requirement already satisfied: smart-open>=5.2.1 in /usr/local/lib/python3.13/dist-packages (from weasel<2.0.0,>=1.0.0->spacy) (8.0.2)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.13/dist-packages (from rich>=13.8.0->typer<1.0.0,>=0.3.0->spacy) (2.21.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.13/dist-packages (from smart-open>=5.2.1->weasel<2.0.0,>=1.0.0->spacy) (2.5.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.13/dist-packages (from markdown-it-py>=2.2.0->rich>=13.8.0->typer<1.0.0,>=0.3.0->spacy) (0.1.2)
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 22.0 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

```
#Nivass C R[241801193]
df = pd.read_csv("Amazon_Fine_Food_Reviews_Practice.csv")

print(df.head())
print(df.tail())
print("Shape:", df.shape)
print(df.dtypes)
print(df.isnull().sum())
print("Duplicates:", df.duplicated().sum())
print(df.describe(include="all"))
```

```
   ReviewID                                               Text  Rating  \
0         1  I love this organic green tea from Amazon. The...       4
1         2  Nature's Bakery makes excellent fig bars; I bo...       5
2         3  The headphones from Sony are comfortable, but ...       1
3         4  Dr. Emily Carter recommended this vitamin supp...       5
4         5  This coffee maker by Philips is worth $89 and ...       1

  ProductCategory Reviewer  ReviewDate
0          Beauty    Aarav  2025-01-01
1     Electronics    Meera  2025-01-02
2     Electronics    Priya  2025-01-03
3           Books    Meera  2025-01-04
4           Books    Rahul  2025-01-05
     ReviewID                                               Text  Rating  \
115       116  The Philips mug is useful. I bought 3 units fo...       5
116       117  The Sony backpack is affordable. I bought 4 un...       2
117       118  The Sony charger is useful. I bought 4 units f...       2
118       119  The Target tea is useful. I bought 3 units for...       5
119       120  The Cetaphil water bottle is useful. I bought ...       5

     ProductCategory Reviewer  ReviewDate
115             Food     Priya  2025-04-26
116           Beauty     Aarav  2025-04-27
117            Books     Aarav  2025-04-28
118             Home     Rahul  2025-04-29
119            Books     Priya  2025-04-30
Shape: (120, 6)
ReviewID             int64
Text                object
Rating               int64
ProductCategory     object
Reviewer            object
ReviewDate          object
dtype: object
ReviewID            0
Text                0
Rating              0
ProductCategory     0
Reviewer            0
ReviewDate          0
dtype: int64
Duplicates: 0
          ReviewID                                               Text  \
count   120.000000                                                120
unique         NaN                                                120
top            NaN  I love this organic green tea from Amazon. The...
freq           NaN                                                  1
mean     60.500000                                                NaN
std      34.785054                                                NaN
min       1.000000                                                NaN
25%      30.750000                                                NaN
50%      60.500000                                                NaN
75%      90.250000                                                NaN
max     120.000000                                                NaN

           Rating ProductCategory Reviewer  ReviewDate
count  120.000000             120      120         120
unique        NaN               5        6         120
top           NaN            Food    Meera  2025-01-01
freq          NaN              29       22           1
mean     3.075000             NaN      NaN         NaN
std      1.432882             NaN      NaN         NaN
min      1.000000             NaN      NaN         NaN
25%      2.000000             NaN      NaN         NaN
50%      3.000000             NaN      NaN         NaN
75%      4.000000             NaN      NaN         NaN
max      5.000000             NaN      NaN         NaN
```

```
#Nivass C R[241801193]
df = df[["Text"]].dropna()
df["Text"] = df["Text"].astype(str).str.strip()
df = df[df["Text"] != ""]
df = df.drop_duplicates(subset="Text")
df = df.head(10000).reset_index(drop=True)

print("Cleaned shape:", df.shape)
print(df.head())
```

```
Cleaned shape: (120, 1)
                                                Text
0  I love this organic green tea from Amazon. The...
1  Nature's Bakery makes excellent fig bars; I bo...
2  The headphones from Sony are comfortable, but ...
3  Dr. Emily Carter recommended this vitamin supp...
4  This coffee maker by Philips is worth $89 and ...
```

```
#Nivass C R[241801193]
nlp = spacy.load("en_core_web_sm")

print("spaCy model loaded successfully")
```

```
spaCy model loaded successfully
```

```
#Nivass C R[241801193]
for review in df["Text"].head(5):
    doc = nlp(review)

    print("\nReview:", review)
    print("Tokens:", [token.text for token in doc])
    print("Lemmas:", [token.lemma_ for token in doc])
    print("POS Tags:", [(token.text, token.pos_) for token in doc])
```

```
Review: I love this organic green tea from Amazon. The package arrived on March 5, 2024, and cost $12.99.
Tokens: ['I', 'love', 'this', 'organic', 'green', 'tea', 'from', 'Amazon', '.', 'The', 'package', 'arrived', 'on', 'March', '5', ',', '2024', ',', 'and', 'cost', '$', '12.99', '.']
Lemmas: ['I', 'love', 'this', 'organic', 'green', 'tea', 'from', 'Amazon', '.', 'the', 'package', 'arrive', 'on', 'March', '5', ',', '2024', ',', 'and', 'cost', '$', '12.99', '.']
POS Tags: [('I', 'PRON'), ('love', 'VERB'), ('this', 'DET'), ('organic', 'ADJ'), ('green', 'ADJ'), ('tea', 'NOUN'), ('from', 'ADP'), ('Amazon', 'PROPN'), ('.', 'PUNCT'), ('The', 'DET'), ('package', 'NOUN'), ('arrived', 'VERB'), ('on', 'ADP'), ('March', 'PROPN'), ('5', 'NUM'

Review: Nature's Bakery makes excellent fig bars; I bought 3 boxes yesterday.
Tokens: ['Nature', "'s", 'Bakery', 'makes', 'excellent', 'fig', 'bars', ';', 'I', 'bought', '3', 'boxes', 'yesterday', '.']
Lemmas: ['nature', "'s", 'Bakery', 'make', 'excellent', 'fig', 'bar', ';', 'I', 'buy', '3', 'box', 'yesterday', '.']
POS Tags: [('Nature', 'NOUN'), ("'s", 'PART'), ('Bakery', 'PROPN'), ('makes', 'VERB'), ('excellent', 'ADJ'), ('fig', 'NOUN'), ('bars', 'NOUN'), (';', 'PUNCT'), ('I', 'PRON'), ('bought', 'VERB'), ('3', 'NUM'), ('boxes', 'NOUN'), ('yesterday', 'NOUN'), ('.', 'PUNCT')]

Review: The headphones from Sony are comfortable, but the battery lasted only 4 hours.
Tokens: ['The', 'headphones', 'from', 'Sony', 'are', 'comfortable', ',', 'but', 'the', 'battery', 'lasted', 'only', '4', 'hours', '.']
Lemmas: ['the', 'headphone', 'from', 'Sony', 'be', 'comfortable', ',', 'but', 'the', 'battery', 'last', 'only', '4', 'hour', '.']
POS Tags: [('The', 'DET'), ('headphones', 'NOUN'), ('from', 'ADP'), ('Sony', 'PROPN'), ('are', 'AUX'), ('comfortable', 'ADJ'), (',', 'PUNCT'), ('but', 'CCONJ'), ('the', 'DET'), ('battery', 'NOUN'), ('lasted', 'VERB'), ('only', 'ADV'), ('4', 'NUM'), ('hours', 'NOUN'), ('.',

Review: Dr. Emily Carter recommended this vitamin supplement in 2023.
Tokens: ['Dr.', 'Emily', 'Carter', 'recommended', 'this', 'vitamin', 'supplement', 'in', '2023', '.']
Lemmas: ['Dr.', 'Emily', 'Carter', 'recommend', 'this', 'vitamin', 'supplement', 'in', '2023', '.']
POS Tags: [('Dr.', 'PROPN'), ('Emily', 'PROPN'), ('Carter', 'PROPN'), ('recommended', 'VERB'), ('this', 'DET'), ('vitamin', 'NOUN'), ('supplement', 'NOUN'), ('in', 'ADP'), ('2023', 'NUM'), ('.', 'PUNCT')]

Review: This coffee maker by Philips is worth $89 and arrived in two days.
Tokens: ['This', 'coffee', 'maker', 'by', 'Philips', 'is', 'worth', '$', '89', 'and', 'arrived', 'in', 'two', 'days', '.']
Lemmas: ['this', 'coffee', 'maker', 'by', 'Philips', 'be', 'worth', '$', '89', 'and', 'arrive', 'in', 'two', 'day', '.']
POS Tags: [('This', 'DET'), ('coffee', 'NOUN'), ('maker', 'NOUN'), ('by', 'ADP'), ('Philips', 'PROPN'), ('is', 'AUX'), ('worth', 'ADJ'), ('$', 'SYM'), ('89', 'NUM'), ('and', 'CCONJ'), ('arrived', 'VERB'), ('in', 'ADP'), ('two', 'NUM'), ('days', 'NOUN'), ('.', 'PUNCT')]
```

```
#Nivass C R[241801193]
counts = Counter()

for review in df["Text"]:
    doc = nlp(review)
    counts.update(token.pos_ for token in doc)

doc = nlp(df["Text"][0])

print("Nouns:", [t.text for t in doc if t.pos_ in ["NOUN", "PROPN"]])
print("Verbs:", [t.text for t in doc if t.pos_ == "VERB"])
print("Adjectives:", [t.text for t in doc if t.pos_ == "ADJ"])
print("Adverbs:", [t.text for t in doc if t.pos_ == "ADV"])

for pos in ["NOUN", "PROPN", "VERB", "AUX", "ADJ", "ADV"]:
    print(pos, ":", counts[pos])
```

```
Nouns: ['tea', 'Amazon', 'package', 'March']
Verbs: ['love', 'arrived', 'cost']
Adjectives: ['organic', 'green']
Adverbs: []
NOUN : 640
PROPN : 122
VERB : 316
AUX : 97
ADJ : 115
ADV : 5
```

```
#Nivass C R[241801193]
entities = []

for i, review in enumerate(df["Text"], start=1):
    doc = nlp(review)

    for ent in doc.ents:
        entities.append({
            "ReviewID": i,
            "Review": review,
            "Entity": ent.text,
            "Entity Label": ent.label_
        })

entities_df = pd.DataFrame(entities)

print(entities_df.head(20))
```

```
    ReviewID                                             Review  \
0          1  I love this organic green tea from Amazon. The...
1          1  I love this organic green tea from Amazon. The...
2          1  I love this organic green tea from Amazon. The...
3          2  Nature's Bakery makes excellent fig bars; I bo...
4          2  Nature's Bakery makes excellent fig bars; I bo...
5          2  Nature's Bakery makes excellent fig bars; I bo...
6          3  The headphones from Sony are comfortable, but ...
7          3  The headphones from Sony are comfortable, but ...
8          4  Dr. Emily Carter recommended this vitamin supp...
9          4  Dr. Emily Carter recommended this vitamin supp...
10         5  This coffee maker by Philips is worth $89 and ...
11         5  This coffee maker by Philips is worth $89 and ...
12         5  This coffee maker by Philips is worth $89 and ...
13         6  The chocolate tastes delicious and the 500 gra...
14         7  Apple customer service replaced my damaged dev...
15         7  Apple customer service replaced my damaged dev...
16         8  I ordered two bottles of olive oil from Califo...
17         8  I ordered two bottles of olive oil from Califo...
18         9  The book by John Smith explains machine learni...
19        10  These running shoes cost 75 dollars and were d...

        Entity Entity Label
0       Amazon          ORG
1   March 5, 2024       DATE
2           12.99      MONEY
3             fig          ORG
4               3    CARDINAL
5       yesterday       DATE
6            Sony          ORG
7     only 4 hours       TIME
8     Emily Carter     PERSON
9             2023       DATE
10         Philips          ORG
11              89      MONEY
12        two days       DATE
13          500 gram   QUANTITY
14             Apple          ORG
15      January 12       DATE
16             two    CARDINAL
17      California          GPE
18      John Smith     PERSON
19      75 dollars      MONEY
```

```
#Nivass C R[241801193]
def extract_information(review):
    doc = nlp(review)

    return {
        "Tokens": [t.text for t in doc],
        "Lemmas": [t.lemma_ for t in doc],
        "POS Tags": [t.pos_ for t in doc],
        "Named Entities": [e.text for e in doc.ents],
        "Entity Labels": [e.label_ for e in doc.ents]
    }

reviews = [
    "Ravi bought a Sony speaker for $45 in Chennai.",
    "Priya purchased two bottles of shampoo yesterday.",
    "The Samsung phone arrived on January 12, 2025."
]

for review in reviews:
    print("\nReview:", review)
    print(extract_information(review))
```

```
Review: Ravi bought a Sony speaker for $45 in Chennai.
{'Tokens': ['Ravi', 'bought', 'a', 'Sony', 'speaker', 'for', '$', '45', 'in', 'Chennai', '.'], 'Lemmas': ['ravi', 'buy', 'a', 'Sony', 'speaker', 'for', '$', '45', 'in', 'Chennai', '.'], 'POS Tags': ['NOUN', 'VERB', 'DET', 'PROPN', 'NOUN', 'ADP', 'SYM', 'NUM', 'ADP', 'PROPN', 'PUNCT'], 'Named Entities': ['Ravi', 'Sony', '45', 'Chennai'], 'Entity Labels': ['ORG', 'ORG', 'M

Review: Priya purchased two bottles of shampoo yesterday.
{'Tokens': ['Priya', 'purchased', 'two', 'bottles', 'of', 'shampoo', 'yesterday', '.'], 'Lemmas': ['Priya', 'purchase', 'two', 'bottle', 'of', 'shampoo', 'yesterday', '.'], 'POS Tags': ['PROPN', 'VERB', 'NUM', 'NOUN', 'ADP', 'NOUN', 'NOUN', 'PUNCT'], 'Named Entities': ['two', 'yesterday'], 'Entity Labels': ['CARDINAL', 'DATE']}

Review: The Samsung phone arrived on January 12, 2025.
{'Tokens': ['The', 'Samsung', 'phone', 'arrived', 'on', 'January', '12', ',', '2025', '.'], 'Lemmas': ['the', 'Samsung', 'phone', 'arrive', 'on', 'January', '12', ',', '2025', '.'], 'POS Tags': ['DET', 'PROPN', 'NOUN', 'VERB', 'ADP', 'PROPN', 'NUM', 'PUNCT', 'NUM', 'PUNCT'], 'Named Entities': ['Samsung', 'January 12, 2025'], 'Entity Labels': ['ORG', 'DATE']}
```

```python
#Nivass C R[241801193]
entities_df.to_csv("ex5_extracted_entities.csv", index=False)

results = []

for i, review in enumerate(df["Text"], start=1):
    info = extract_information(review)

    results.append({
        "ReviewID": i,
        "Review": review,
        "Tokens": ", ".join(info["Tokens"]),
        "Lemmas": ", ".join(info["Lemmas"]),
        "POS Tags": ", ".join(info["POS Tags"]),
        "Named Entities": ", ".join(info["Named Entities"]),
        "Entity Labels": ", ".join(info["Entity Labels"])
    })

result_df = pd.DataFrame(results)
result_df.to_csv("ex5_review_information.csv", index=False)

print("Files saved successfully!")
print(result_df.head())
```

```
Files saved successfully!
   ReviewID                                             Review  \
0         1  I love this organic green tea from Amazon. The...
1         2  Nature's Bakery makes excellent fig bars; I bo...
2         3  The headphones from Sony are comfortable, but ...
3         4  Dr. Emily Carter recommended this vitamin supp...
4         5  This coffee maker by Philips is worth $89 and ...

                                              Tokens  \
0  I, love, this, organic, green, tea, from, Amaz...
1  Nature, 's, Bakery, makes, excellent, fig, bar...
2  The, headphones, from, Sony, are, comfortable,...
3  Dr., Emily, Carter, recommended, this, vitamin...
4  This, coffee, maker, by, Philips, is, worth, $...

                                              Lemmas  \
0  I, love, this, organic, green, tea, from, Amaz...
1  nature, 's, Bakery, make, excellent, fig, bar,...
2  the, headphone, from, Sony, be, comfortable, ,...
3  Dr., Emily, Carter, recommend, this, vitamin, ...
4  this, coffee, maker, by, Philips, be, worth, $...

                                            POS Tags  \
0  PRON, VERB, DET, ADJ, ADJ, NOUN, ADP, PROPN, P...
1  NOUN, PART, PROPN, VERB, ADJ, NOUN, NOUN, PUNC...
2  DET, NOUN, ADP, PROPN, AUX, ADJ, PUNCT, CCONJ,...
3  PROPN, PROPN, PROPN, VERB, DET, NOUN, NOUN, AD...
4  DET, NOUN, NOUN, ADP, PROPN, AUX, ADJ, SYM, NU...

                 Named Entities        Entity Labels
0  Amazon, March 5, 2024, 12.99     ORG, DATE, MONEY
1            fig, 3, yesterday  ORG, CARDINAL, DATE
2           Sony, only 4 hours            ORG, TIME
3           Emily Carter, 2023          PERSON, DATE
4        Philips, 89, two days     ORG, MONEY, DATE
```